Universidad Tecnológica del Sureste de Veracruz
Ingeniería en Desarrollo y Gestión de Software

Desarrollo Móvil Integral
Grupo 1001



RESULTADO DE APRENDIZAJE · EVIDENCIA

Flashly v0.4.0

Documentación de la aplicación móvil: repositorio, metodología ágil, arquitectura, patrones, frameworks, pruebas, seguridad, APIs de terceros y servicios en la nube

EQUIPO

José Jamin García García
Jorge Israel Aguilera Rueda
Bárbara Alejandra Díaz Pérez

DOCENTE

Mtro. Isaías Gallegos Salvador

FECHA

28 de octubre de 2026

Nanchital, Veracruz

github.com/Jamgargar04/flashly

Contenido

Flashly v0.4.0 · 15 secciones · 38 figuras

01	Datos generales	3
02	Resultado de aprendizaje y cumplimiento	3
03	Caso de estudio	4
04	Repositorio de código fuente	6
05	Metodología ágil	9
06	Arquitectura de software	11
07	Patrones de diseño y SOLID	13
08	Frameworks y tecnologías	17
09	Esquema de pruebas	19
10	Mecanismos de seguridad	24
11	APIs de terceros	30
12	Servicios en la nube	33
13	Aplicación en funcionamiento	37
14	Enlace del repositorio en funcionamiento	45
15	Conclusiones	47

Las figuras se numeran en el orden en que aparecen. Las pantallas de la app son de la compilación web en modo demostración y de un teléfono Android con el APK; las de código, terminal y consolas son de los archivos, comandos y servicios reales del proyecto.

01 Datos generales

Flashly es una aplicación móvil para la comunidad de la UTSV construida con React Native, Expo y Firebase. Este documento demuestra, con capturas de la versión v0.4.0, cómo la app cubre cada punto del resultado de aprendizaje de la materia.

Dato	Valor
Institución	Universidad Tecnológica del Sureste de Veracruz (UTSV)
Carrera	Ingeniería en Desarrollo y Gestión de Software
Materia	Desarrollo Móvil Integral
Docente	Mtro. Isaías Gallegos Salvador
Equipo	José Jamin García García, Jorge Israel Aguilera Rueda y Bárbara Alejandra Díaz Pérez · grupo 1001
Versión documentada	v0.4.0 · commit `fcc738c` · rama `develop` · 15 de octubre de 2026
Repositorio	github.com/Jamgargar04/flashly
Paquete Android	`mx.edu.utsv.flashly`

> **Sobre las capturas.** Las pantallas de la app se capturaron de la compilación web de la versión v0.4.0 (React Native Web) en modo demostración, con un tamaño de teléfono de 390 × 844 px. Las capturas de código y de terminal salen de los archivos y comandos reales del repositorio en esa misma versión. Las capturas de las consolas (Firebase, Cloudinary, GitHub y Expo) y las del teléfono Android las tomó el equipo el 28 de octubre de 2026.

02 Resultado de aprendizaje y cumplimiento

Flashly v0.4.0 cubre los cuatro elementos que pide el resultado de aprendizaje y cada requisito del enunciado tiene evidencia propia en este documento.

> *Desarrolla una aplicación móvil utilizando un repositorio de código fuente, con base en un caso de estudio de acuerdo a la metodología ágil, la arquitectura de software, los patrones de diseño, frameworks y esquema de pruebas seleccionados, que incluya: mecanismos de seguridad, API's de terceros, servicios en la nube y enlace del repositorio en funcionamiento.*

Requisito	Cómo lo cubre Flashly	Sección
Repositorio de código fuente	GitHub público con 17 commits, ramas `main` y `develop` e integración continua en cada push	04
Caso de estudio	Poca convivencia entre estudiantes de distintas carreras y grupos de la UTSV	03
Metodología ágil	Scrum adaptado: backlog de historias de usuario, cinco incrementos, definición de terminado y CI	05
Arquitectura de software	Clean Architecture en cuatro capas con puertos y adaptadores	06
Patrones de diseño	Repository, Strategy, Adapter, Factory, Singleton, Observer, inyección de dependencias y Store	07
Frameworks	Expo SDK 57, React Native 0.86, Expo Router, Zustand, Zod y Firebase JS SDK 12	08
Esquema de pruebas	56 pruebas automáticas con Jest, TypeScript estricto, GitHub Actions y un plan de 10 pruebas manuales	09
Mecanismos de seguridad	Correo institucional verificado, reglas de Firestore y Storage, HTTPS, secretos fuera del repositorio y retención de 24 h	10
APIs de terceros	Cloudinary Upload API, Google Cloud Vision y APIs nativas de Expo	11
Servicios en la nube	Firebase Authentication, Cloud Firestore, Cloud Functions y Expo EAS Build	12
Enlace del repositorio en funcionamiento	github.com/Jamgargar04/flashly, última ejecución de CI (#14) exitosa y APK instalable desde Expo	14

03 Caso de estudio

El caso de estudio es la poca convivencia entre estudiantes de la UTSV: se pasan años en el mismo campus y cada quien conoce solo a su grupo. Flashly lo ataca con una dinámica diaria y espontánea que muestra quién más está en el campus y qué está haciendo.

Propuesta. Una vez al día, a una hora sorpresa, todos reciben el mismo aviso y tienen 2 minutos para compartir una foto con la cámara trasera y otra con la frontal. Para ver los momentos de los demás, primero hay que publicar el propio. Alrededor de esa dinámica, la v0.4.0 suma chat del campus, eventos, búsqueda de compañeros y lugares con movimiento.



Figura 1. Las tres pantallas de bienvenida resumen el caso de estudio: conocer a tu generación, un aviso sorpresa al día y "si no publicas, no ves".

Actores

- **Alumno:** cuenta con correo `@alumnos.utsv.edu.mx`; publica, reacciona, sigue a otros y usa el chat.
- **Maestro:** cuenta con correo `@utsv.edu.mx`; mismas funciones que un alumno, sin grupo.
- **Administrador:** crea y borra eventos, modera momentos y mensajes. El permiso solo se otorga desde la base de datos (Figura 26).

Reglas del negocio

Implementadas en `src/domain` y repetidas en las reglas del servidor:

1. La ventana del flash cae entre las 9:00 y las 20:00, dura 2 minutos y es la misma en todos los teléfonos, porque se calcula con un hash de la fecha.
2. Cada momento lleva dos fotos: primero la cámara trasera y, 1.1 s después, la frontal.
3. Solo se publica un momento por persona al día: el identificador siempre es `día_usuario`.
4. Si no publicas, no ves: el feed y los lugares se desbloquean al compartir el momento propio.
5. Publicar después de los 2 minutos se permite, pero el momento queda marcado como "Tarde".

6. Los momentos dejan de mostrarse 24 horas después de publicarse.
7. Solo entran cuentas con correo institucional verificado.

04 Repositorio de código fuente

El código vive en un repositorio público de GitHub, Jamgargar04/flashly, con 17 commits, dos ramas remotas y GitHub Actions ejecutando tipos y pruebas en cada push. La versión v0.4.0 está en la rama `develop` (commit `fcc738c`).

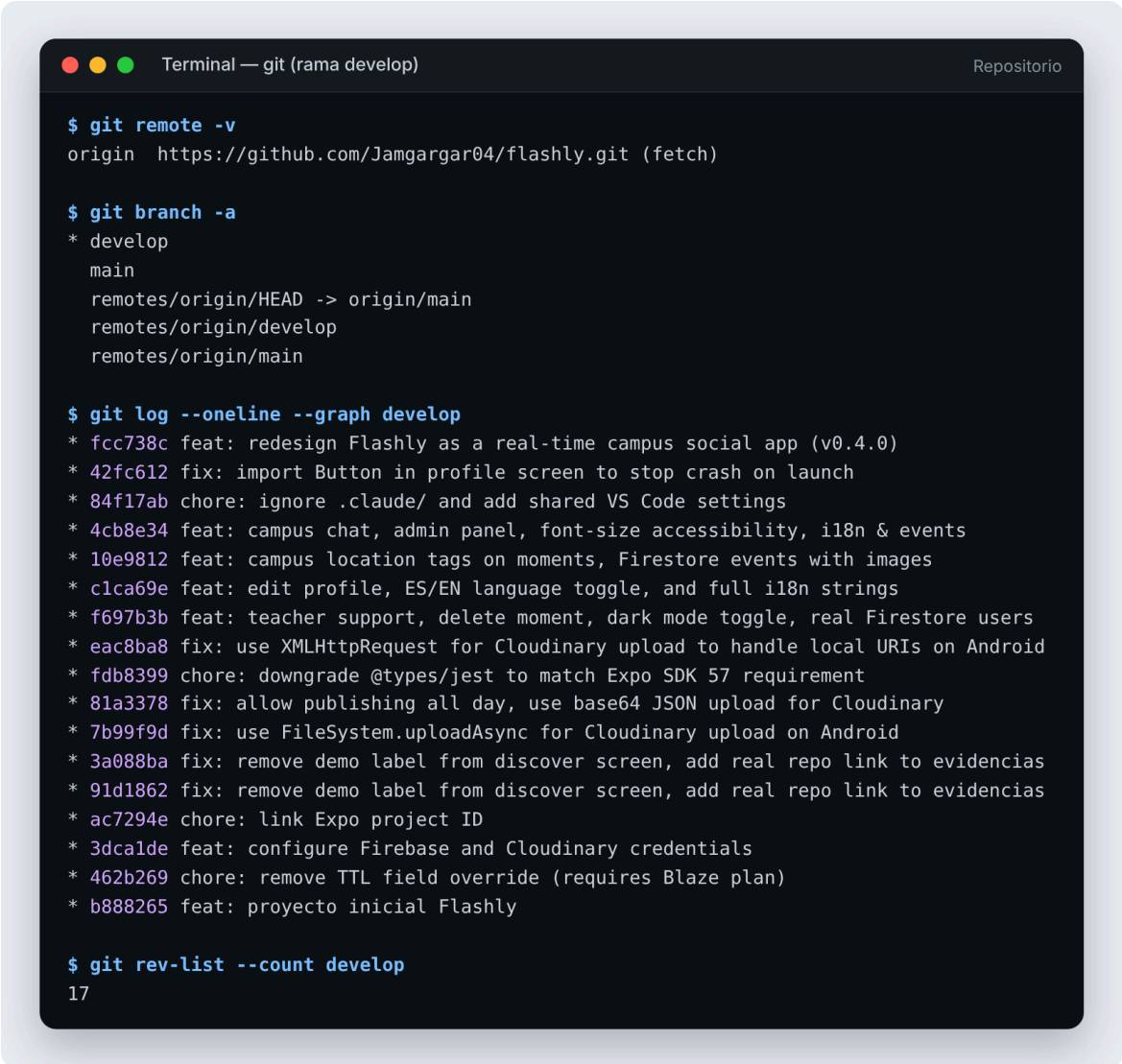


Figura 2. Salida real de git: remoto en GitHub, ramas `main` y `develop` y los 17 commits de `develop`.

Flujo de ramas. `main` guarda la versión estable, `develop` integra el trabajo de cada incremento, y los cambios nuevos se hacen en ramas `feature/<nombre>` o `fix/<nombre>`. La guía de contribución del README pide que cada cambio entre por Pull Request revisado por otro integrante, y la plantilla con la

definición de terminado ya está en el repositorio.

Mensajes de commit. El historial sigue Conventional Commits, así cada cambio dice su tipo:

Tipo	Commits	Ejemplo del historial
`feat:` nueva función	7	`feat: campus chat, admin panel, font-size accessibility, i18n & events`
`fix:` corrección	6	`fix: use XMLHttpRequest for Cloudinary upload to handle local URIs on Android`
`chore:` mantenimiento	4	`chore: link Expo project ID`

Integración continua. El flujo `.github/workflows/ci.yml` instala dependencias con `npm ci`, revisa tipos con `tsc` y corre Jest con cobertura en cada push a `main` o `develop` y en cada Pull Request. Hasta hoy suma 14 ejecuciones; tres fallaron (#7, #11 y #12) y los commits siguientes las corrigieron. La ejecución #14, sobre la v0.4.0, terminó con éxito en 53 s.

.github/workflows/ci.yml · .github/pull_request_template.md — Integración continua

.github/workflows/ci.yml

```yaml
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:

jobs:
  check:
    name: Tipos y pruebas
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run typecheck
      - run: npm test -- --ci --coverage
```

.github/pull_request_template.md

```markdown
## ¿Qué cambia?

<!-- Historia de usuario o tarea del sprint que resuelve este Pull Request. -->

## Definición de terminado

- [ ] Pasa `npm run typecheck` y `npm test`.
- [ ] Probado en un teléfono Android real.
- [ ] No expone claves ni datos personales.
- [ ] Documentado si cambia la arquitectura o la configuración.
```

Figura 3. Flujo de GitHub Actions y plantilla de Pull Request con la definición de terminado.

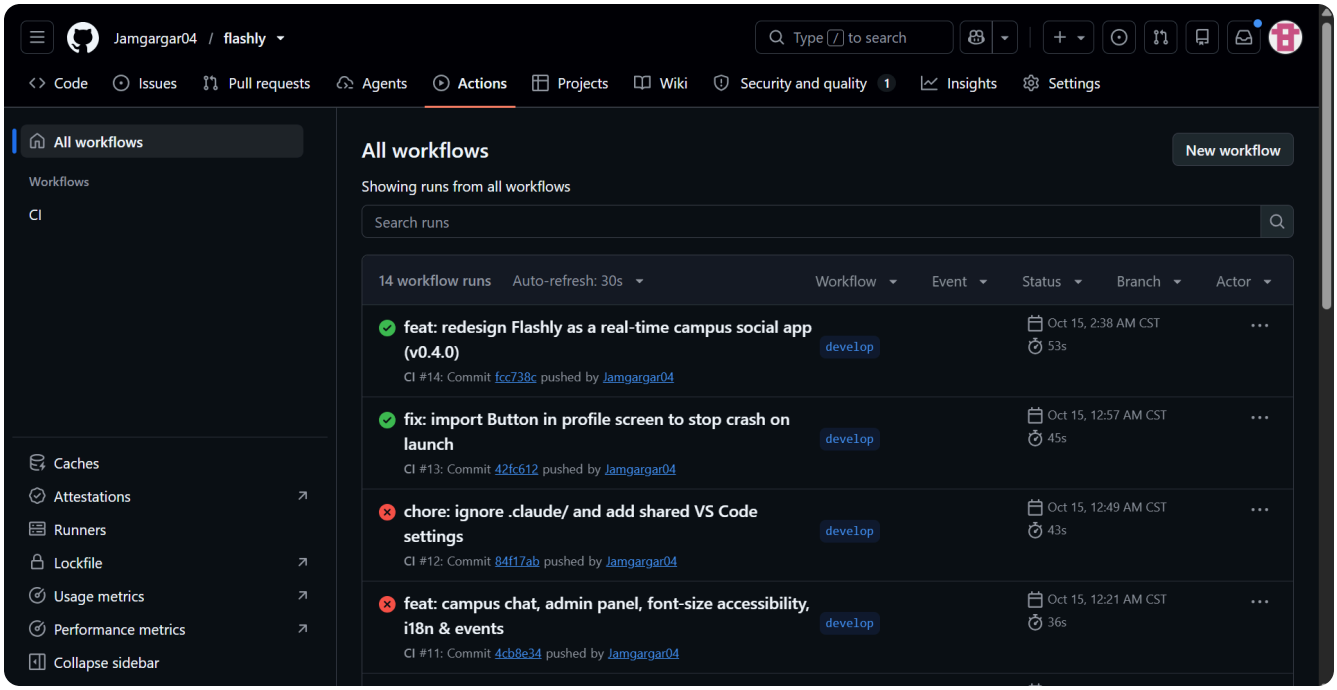


Figura 4. GitHub Actions: 14 ejecuciones del flujo CI. La #14, sobre el commit `fcc738c` de la v0.4.0, pasó en 53 s; las fallidas #11 y #12 se corrigieron en el commit siguiente.

05 Metodología ágil

El proyecto se trabajó con Scrum adaptado a un equipo de tres integrantes: un backlog de historias de usuario priorizadas, entregas en incrementos funcionales y una definición de terminado que se revisa con integración continua. Cada incremento dejó la app funcionando y se puede rastrear en el historial de commits.

Product backlog. Las historias de la v0.4.0 están terminadas; las pendientes vienen de la hoja de ruta hacia producción (`docs/HOJA-DE-RUTA.md`).

ID	Historia de usuario	Prioridad	Estado
HU-01	Como estudiante quiero registrarme con mi correo institucional para que solo entre gente de la UTSV.	Alta	Terminada
HU-02	Como estudiante quiero recibir un aviso sorpresa al día para participar al mismo tiempo que todos.	Alta	Terminada
HU-03	Como estudiante quiero tomar la foto trasera y la frontal en un solo paso para mostrar qué hago y quién lo hace.	Alta	Terminada
HU-04	Como estudiante quiero ver los momentos de los demás solo después de publicar el mío.	Alta	Terminada

ID	Historia de usuario	Prioridad	Estado
HU-05	Como estudiante quiero reaccionar a los momentos y ver las reacciones al instante.	Media	Terminada
HU-06	Como estudiante quiero etiquetar dónde estoy y ver los lugares del campus con movimiento.	Media	Terminada
HU-07	Como estudiante quiero buscar y seguir a mis compañeros.	Media	Terminada
HU-08	Como estudiante quiero platicar con todo el campus en un chat.	Media	Terminada
HU-09	Como administrador quiero publicar eventos con portada para que los alumnos pongan un recordatorio.	Media	Terminada
HU-10	Como estudiante quiero reportar un momento o mensaje inapropiado.	Media	Terminada
HU-11	Como usuario quiero tema oscuro, texto grande e idioma inglés.	Baja	Terminada
HU-12	Como responsable del sistema quiero subidas firmadas a Cloudinary para que nadie use el preset fuera de la app.	Alta	Pendiente
HU-13	Como responsable del sistema quiero probar las reglas de Firestore en el emulador.	Alta	Pendiente
HU-14	Como estudiante quiero notificaciones push enviadas desde el servidor.	Media	Pendiente

Incrementos entregados. Se reconstruyen del historial de `develop`, del más reciente al primero:

Incremento	Fecha	Qué entregó	Commits
5 · v0.4.0	15 oct 2026	Rediseño como red social del campus en tiempo real y corrección de un cierre al abrir el perfil	`fcc738c` `42fc612` `84f17ab`
4 · Funciones sociales	14–15 oct 2026	Maestros, borrar momento, tema oscuro, editar perfil, español e inglés, lugares, eventos, chat y panel de admin	`f697b3b` `c1ca69e` `10e9812` `4cb8e34`
3 · Estabilización en Android	14 oct 2026	Subida de fotos a Cloudinary desde rutas locales de Android y publicación todo el día	`7b99f9d` `81a3378` `fdb8399` `eac8ba8`
2 · Conexión a la nube	14 oct 2026	Credenciales de Firebase y Cloudinary, proyecto de Expo y enlace del repositorio en las evidencias	`462b269` `3dca1de` `ac7294e` `91d1862` `3a088ba`
1 · Proyecto inicial	14 oct 2026	Arquitectura en capas, dominio, modo demostración, pruebas y CI	`b888265`

Definición de terminado. Una historia se da por terminada cuando pasa `npm run typecheck` y `npm test`, se probó en un teléfono Android real, no expone claves ni datos personales y queda documentada si cambia la arquitectura o la configuración. Esta lista está en `.github/pull_request_template.md` (Figura 3).

Inspección y adaptación. La CI funciona como revisión automática de cada incremento: cuando una ejecución falló (#7, #11 y #12), el siguiente commit corrigió el problema antes de seguir con nuevas historias.

06 Arquitectura de software

Flashly usa Clean Architecture en cuatro capas: las reglas del negocio no dependen de Expo, Firebase ni Cloudinary. Gracias a eso la misma app funciona en modo demostración sin internet y en modo nube cambiando solo la configuración.

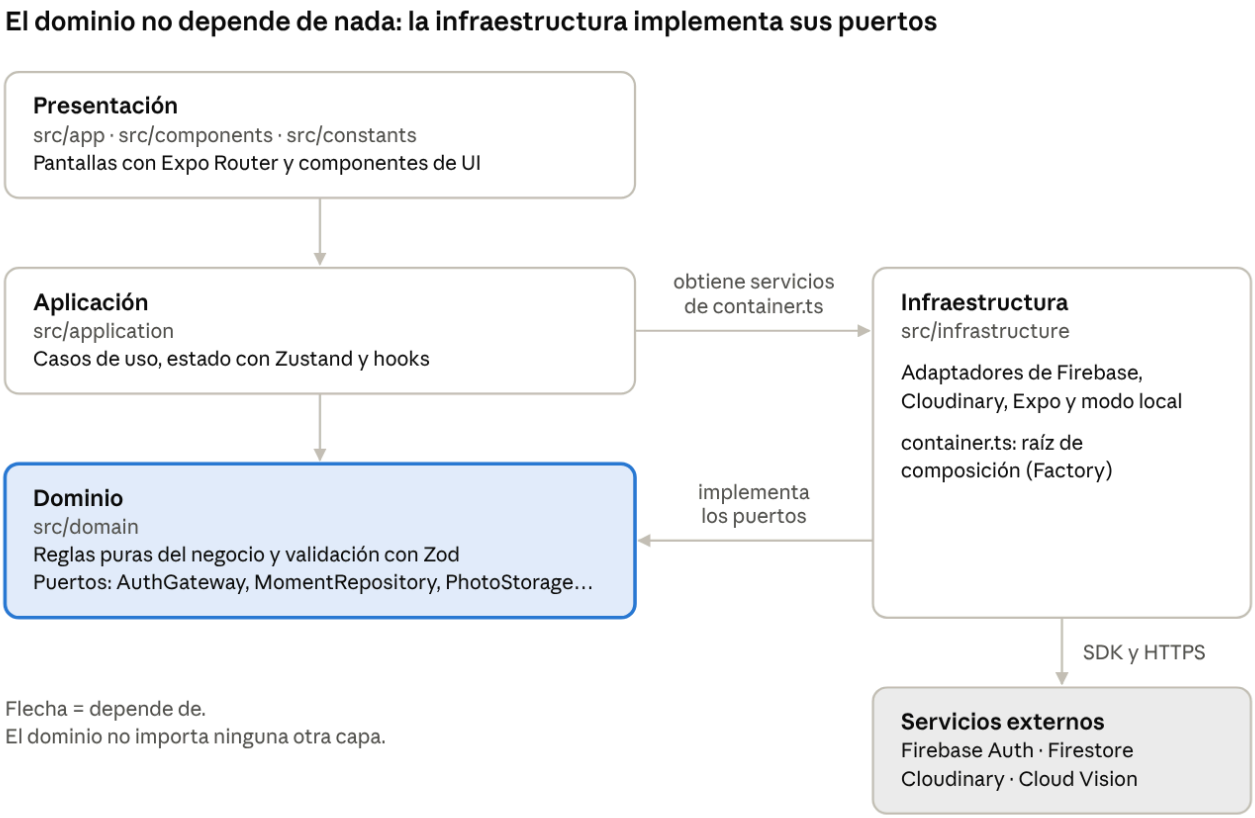


Figura 5. Capas y dependencias. Solo `infrastructure/container.ts` conoce las clases concretas; el resto del código depende de los puertos del dominio.



Figura 6. La estructura de `src/` refleja las cuatro capas.

Flujo de ejemplo: publicar un momento

1. La pantalla `capture.tsx` toma las dos fotos y llama al hook `usePublishMoment()` (presentación).
2. El caso de uso `createPublishMoment` revisa con `assertCanPublish` que no hayas publicado hoy (dominio).
3. `PhotoCompressor` reduce las fotos a 1080 px y 540 px con Expo Image Manipulator.
4. `PhotoStorage` sube las dos fotos a Cloudinary y devuelve sus URL `https`.
5. `MomentRepository` guarda el momento en Firestore con id `día_usuario`.

6. Las reglas de Firestore validan el id, el autor y las URL antes de aceptar la escritura.

src/domain/ports.ts — Dominio · puertos (interfaces)

```ts
export interface MomentRepository {
  hasPublished(day: string, uid: string): Promise<boolean>;
  save(moment: NewMoment): Promise<void>;
  /** Tu propio momento del día (null si aún no publicas). */
  observeOwn(day: string, uid: string, onChange: (moment: Moment | null) => void, onError: ErrorListener): Unsubscribe;
  observeDay(day: string, windowStart: number, onChange: (moments: Moment[]) => void, onError: ErrorListener): Unsubscribe;
  toggleLike(moment: Moment, uid: string): Promise<void>;
  delete(momentId: string): Promise<void>;
}

export type PhotoTarget = { folder: string; name: string };

export interface PhotoStorage {
  /** Sube una foto local y devuelve la URL pública. */
  upload(localUri: string, target: PhotoTarget): Promise<string>;
}

export interface PhotoCompressor {
  compress(localUri: string, maxWidth: number): Promise<string>;
}
```

Figura 7. Puertos del dominio: interfaces pequeñas que la infraestructura implementa.

07 Patrones de diseño y SOLID

La v0.4.0 aplica ocho patrones de diseño, cada uno en un archivo concreto del repositorio. Juntos permiten cambiar de proveedor o probar sin servicios reales tocando un solo archivo.

Patrón	Dónde	Para qué
Repository	`MomentRepository` → `FirestoreMomentRepository`, `LocalMomentRepository`	Aislar el acceso a datos de la lógica
Strategy	`PhotoStorage` → Cloudinary, Firebase Storage o local	Cambiar el proveedor de fotos sin tocar el caso de uso
Adapter	`FirebaseAuthGateway`, `CloudinaryPhotoStorage`, `ExpoReminderScheduler`	Traducir APIs externas a los puertos y sus errores a mensajes claros
Factory / raíz de composición	`createServices()` en `container.ts`	Construir las implementaciones según la configuración
Singleton	`getFirebase()` en `firebase-app.ts`	Inicializar Firebase una sola vez
Observer	`observe()`, `observeDay()`, `subscribe()` con `onSnapshot`	Actualizar sesión, feed y chat en tiempo real

Patrón	Dónde	Para qué
Inyección de dependencias	`createPublishMoment({ compressor, photos, moments })`	Probar el caso de uso con dobles, sin Firebase ni Cloudinary
Store (Flux)	`useSession`, `useFeed`, `useEvents` con Zustand	Un solo lugar para cada estado global

src/infrastructure/container.ts — Factory · Composition Root

```ts
/**
 * Raíz de composición (Factory): el único lugar que conoce las implementaciones concretas.
 * El resto de la app depende solo de los puertos del dominio.
 */
export const createServices = (config: Env = env): Services => {
  const store = new AsyncStorageStore();
  const device = {
    store,
    compressor: new ExpoPhotoCompressor(),
    picker: new ExpoImagePicker(),
    reminders: new ExpoReminderScheduler(),
  };

  if (!hasFirebase(config)) {
    return {
      ...device,
      mode: 'demo',
      photoProvider: 'local',
      auth: new LocalAuthGateway(store),
      moments: new LocalMomentRepository(store, demoMoments),
      photos: new LocalPhotoStorage(),
      directory: new LocalCampusDirectory(),
      social: new LocalSocialGraph(store),
      events: new LocalEventRepository(),
      chat: new LocalChatRepository(),
      reports: new LocalReportRepository(),
    };
  }

  const firebase = getFirebase(config.firebase);
  const useCloudinary = hasCloudinary(config);
  return {
    ...device,
    mode: 'cloud',
    photoProvider: useCloudinary ? 'cloudinary' : 'firebase',
    auth: new FirebaseAuthGateway(firebase.auth, firebase.db),
    moments: new FirestoreMomentRepository(firebase.db),
    photos: useCloudinary ? new CloudinaryPhotoStorage(config.cloudinary) : new FirebasePhotoStorage(firebase.storage),
    directory: new FirestoreCampusDirectory(firebase.db),
    social: new FirestoreSocialGraph(firebase.db),
    events: new FirestoreEventRepository(firebase.db),
    chat: new FirestoreChatRepository(firebase.db),
    reports: new FirestoreReportRepository(firebase.db),
  };
};
```

Figura 8. `createServices()` elige las implementaciones: sin Firebase arma el modo demostración; con Firebase usa Cloudinary o Firebase Storage para las fotos (Strategy).

src/application/publish-moment.ts — Caso de uso · inyección de dependencias

```ts
/**
 * Caso de uso "Publicar mi momento": valida la regla de negocio, comprime, sube las dos fotos
 * y guarda el momento. Recibe sus dependencias (inyección), así se prueba sin Firebase ni Cloudinary.
 * `onProgress` permite mostrar en qué paso va.
 */
export const createPublishMoment = ({ compressor, photos, moments }: PublishMomentDeps) =>
  async (input: PublishMomentInput, onProgress?: (step: PublishStep) => void): Promise<void> => {
    const { author, day, phase } = input;
    assertCanPublish(phase, await moments.hasPublished(day, author.uid));

    onProgress?.('compress');
    const [back, front] = await Promise.all([
      compressor.compress(input.backUri, PHOTO_WIDTH.back),
      compressor.compress(input.frontUri, PHOTO_WIDTH.front),
    ]);

    onProgress?.('upload');
    const folder = `flashly/${day}/${author.uid}`;
    const [backUrl, frontUrl] = await Promise.all([
      photos.upload(back, { folder, name: 'back' }),
      photos.upload(front, { folder, name: 'front' }),
    ]);

    onProgress?.('save');
    await moments.save({
      id: momentId(day, author.uid),
      day,
      authorId: author.uid,
      authorName: author.name,
      authorGroup: author.group,
      authorPhoto: author.photoUrl,
      backUrl,
      frontUrl,
      caption: sanitizeCaption(input.caption),
      location: input.location,
      late: isLate(phase),
    });
  };
```

Figura 9. El caso de uso recibe sus dependencias por parámetro, por eso se prueba sin servicios reales.

src/infrastructure/firebase/firebase-app.ts — Singleton · Firebase

```ts
let instance: FirebaseServices | null = null;

/** Singleton: Firebase se inicializa una sola vez y la sesión persiste en el teléfono. */
export const getFirebase = (config: Env['firebase']): FirebaseServices => {
  if (instance) return instance;
  const app = getApps().length ? getApp() : initializeApp(config);
  let auth: Auth;
  try {
    auth = initializeAuth(app, { persistence: getReactNativePersistence(AsyncStorage) });
  } catch {
    auth = getAuth(app);
  }
  instance = { app, auth, db: getFirestore(app), storage: getStorage(app) };
  return instance;
};
```

Figura 10. Singleton: Firebase se inicializa una vez y la sesión persiste en el teléfono con AsyncStorage.

src/infrastructure/firebase/firestore-moment-repository.ts — Repository · Firestore en tiempo real

```ts
async save(moment: NewMoment): Promise<void> {
  await setDoc(doc(this.db, COLLECTION, moment.id), {
    authorId: moment.authorId,
    authorName: moment.authorName,
    authorGroup: moment.authorGroup,
    authorPhoto: moment.authorPhoto ?? null,
    windowId: moment.day,
    backUrl: moment.backUrl,
    frontUrl: moment.frontUrl,
    caption: moment.caption,
    location: moment.location ?? null,
    late: moment.late,
    likes: [],
    createdAt: serverTimestamp(),
    // Campo de la política TTL de Firestore: borra el documento a las 24 h.
    expiresAt: Timestamp.fromMillis(Date.now() + MOMENT_TTL_MS),
  });
}
…
observeDay(day: string, _windowStart: number, onChange: (moments: Moment[]) => void, onError: ErrorListener): Unsubscribe {
  const q = query(collection(this.db, COLLECTION), where('windowId', '==', day), orderBy('createdAt', 'desc'), limit(100));
  return onSnapshot(
    q,
    (snap) => {
      const now = Date.now();
      onChange(snap.docs.map((d) => toMoment(d, now)).filter((m): m is Moment => m !== null));
    },
    onError,
  );
}
```

Figura 11. Repository con Observer: `onSnapshot` empuja cada cambio del feed a la pantalla al instante.

Principios SOLID

Principio	Cómo se aplica
S · Responsabilidad única	Cada adaptador hace una cosa: `CloudinaryPhotoStorage` solo sube fotos y las pantallas solo muestran.

Principio	Cómo se aplica
O · Abierto/cerrado	Un nuevo proveedor de fotos es una clase nueva que implementa `PhotoStorage`; el caso de uso no cambia.
L · Sustitución de Liskov	`LocalMomentRepository` y `FirestoreMomentRepository` son intercambiables en pantallas y pruebas.
I · Segregación de interfaces	Puertos pequeños (`PhotoCompressor`, `PhotoStorage`, `KeyValueStore`) en vez de un servicio gigante.
D · Inversión de dependencias	Los casos de uso dependen de interfaces del dominio, no de Firebase ni de Cloudinary.

08 Frameworks y tecnologías

Flashly está escrito en TypeScript sobre Expo SDK 57 y React Native 0.86, con una sola base de código para Android, iOS y web. Las versiones salen de `package.json` de la v0.4.0.

Tecnología	Versión	Uso en Flashly
Expo SDK	~57.0.29	Plataforma, módulos nativos y compilación
React Native	0.86.3	Interfaz nativa en Android e iOS
React	19.2.3	Componentes y React Compiler activado
TypeScript (modo estricto)	~6.0.3	Tipado estático de todo el proyecto
Expo Router	~57.0.27	Navegación por archivos: pestañas, modales y rutas dinámicas
Zustand	^5.0.15	Estado global (sesión, feed, chat, eventos)
Zod	^4.6.7	Validación de formularios y reglas del dominio
Firebase JS SDK	^12.19.0	Authentication, Firestore y Storage
React Native Reanimated	4.5.1	Animaciones fluidas
expo-camera, expo-image-manipulator, expo-image-picker	~57.0	Cámara doble, compresión y galería
expo-notifications	~57.0.23	Aviso del flash y recordatorios de eventos
AsyncStorage	^2.2.0	Sesión y preferencias en el teléfono
Jest + jest-expo	^29.7.0 · ~57.0.5	Pruebas automáticas

Tecnología	Versión	Uso en Flashly
Cloud Functions (firebase-functions, firebase-admin)	^7.4.0 · ^14.5.0	Tareas del servidor y moderación
EAS Build	CLI actual	Generar el APK firmado
GitHub Actions	setup-node@v4, Node 22	Integración continua



Figura 12. Dependencias principales y scripts del proyecto (`typecheck`, `test`, `test:coverage`, `check`).

Decisiones de selección

Documentadas en `docs/ARQUITECTURA.md`:

- **Firestore:** tiempo real, SDK para React Native, plan gratuito y reglas de seguridad integradas; a cambio, consultas más limitadas que SQL.
- **Cloudinary para fotos:** API gratuita sin tarjeta, mientras que Firebase Storage en proyectos nuevos exige el plan Blaze.
- **Zustand y Zod:** librerías probadas en lugar de estado y validación hechos a mano.
- **Modo demostración:** permite desarrollar y presentar sin cuentas en la nube; a cambio, cada puerto tiene dos implementaciones.

09 Esquema de pruebas

La v0.4.0 pasa 56 pruebas automáticas en 9 suites de Jest, compila sin errores con TypeScript estricto y GitHub Actions repite ambas revisiones en cada push. El esquema sigue una pirámide: muchas pruebas unitarias rápidas, pruebas de integración de reglas como siguiente paso y pocas pruebas manuales del flujo completo en el teléfono.

Nivel	Herramienta	Estado en v0.4.0
Unitarias de dominio y aplicación	Jest con dobles de prueba (inyección de dependencias)	56 pruebas, todas pasan
Revisión de tipos	`tsc --noEmit` en modo estricto	Sin errores
Integración continua	GitHub Actions (`ci.yml`)	Ejecución #14 exitosa
Integración de reglas de Firestore	Emulador con `@firebase/rules-unit-testing`	Pendiente (hoja de ruta)
Manuales en teléfono	Plan de 10 casos en debug y release	Ver tabla de casos manuales

Suites automáticas

Suite	Capa	Pruebas	Qué valida
`domain/flash-window.test.ts`	Dominio	8	Misma ventana en todos los teléfonos, rango 9:00–20:00, duración de 2 min, fases en los límites exactos
`domain/event.test.ts`	Dominio	8	Estado de eventos, calendario del formulario, rechazo de títulos cortos y fechas pasadas
`domain/social.test.ts`	Dominio	8	Chat, lugares con movimiento, sugerencias de personas y filtro del feed

Suite	Capa	Pruebas	Qué valida
`domain/student.test.ts`	Dominio	7	Correo institucional, contraseña de 8 caracteres, nombre y correos de maestros
`domain/moment.test.ts`	Dominio	5	Un momento por día, comentario limpio, reacciones sin mutar el original
`application/publish-moment.test.ts`	Aplicación	4	Comprime, sube y guarda; marca tarde; no sube si ya publicaste; no guarda si falla la subida
`application/publish-event.test.ts`	Aplicación	4	Publicar evento con o sin portada y cambiar foto de perfil
`infrastructure/local-adapters.test.ts`	Infraestructura	8	Modo demostración: momentos, sesión local y tiempo real
`infrastructure/cloudinary-photo-storage.test.ts`	Infraestructura	4	API de Cloudinary con `fetch` simulado: preset, URL segura y errores

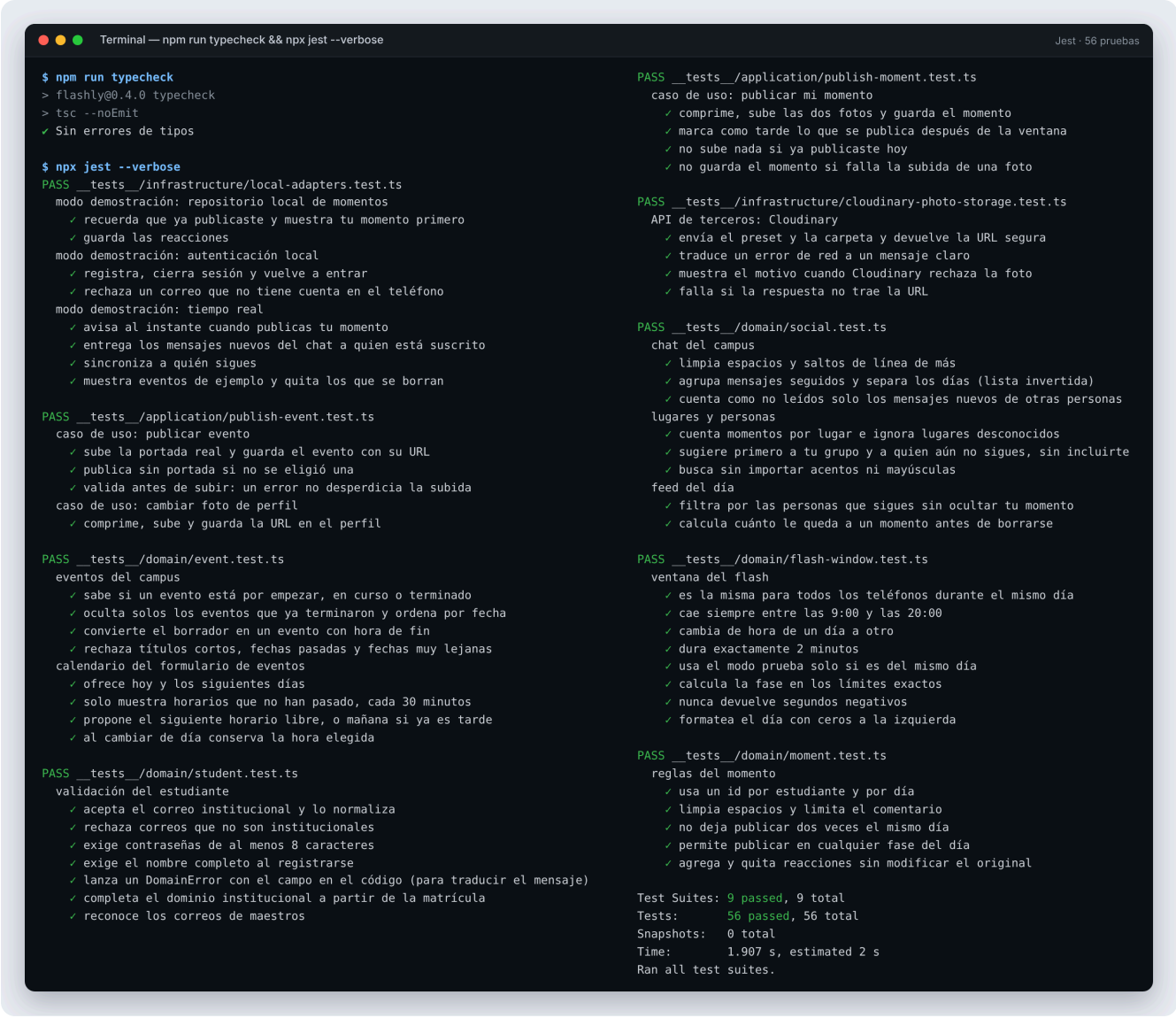
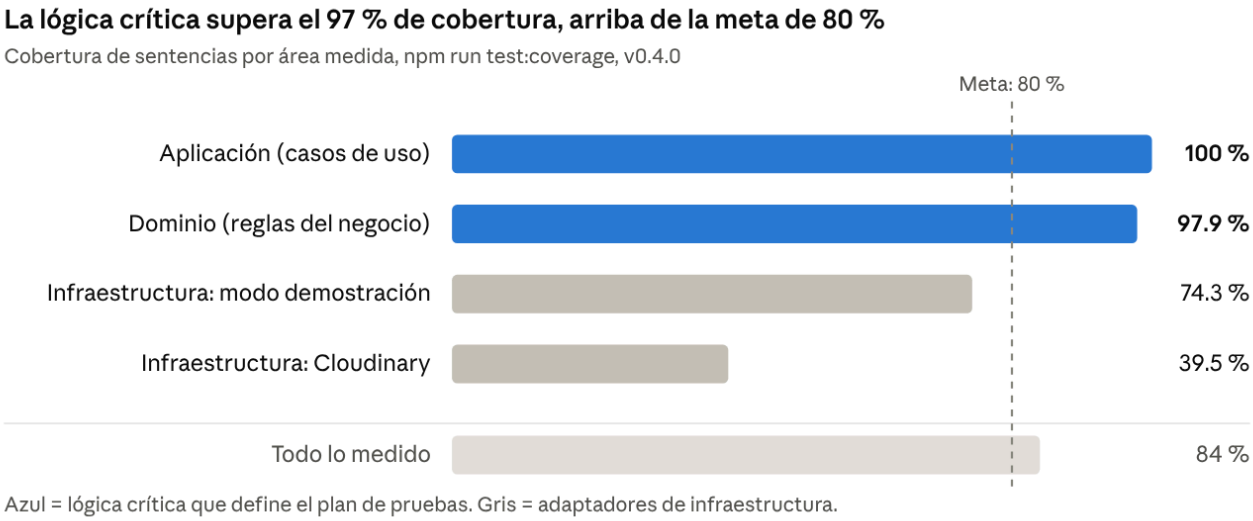


Figura 13. Revisión de tipos sin errores y las 56 pruebas en verde, ejecutadas sobre la v0.4.0.



Gráfica 1. Cobertura de sentencias por área (`npm run test:coverage`, v0.4.0, 28 oct 2026).

La cobertura se mide sobre la lógica crítica que define `collectCoverageFrom` en `package.json`. El adaptador de Cloudinary queda en 39 % porque la ruta nativa con `XMLHttpRequest` solo se ejecuta en el teléfono; la ruta con `fetch` sí está probada.

Terminal — npm run test:coverage

Cobertura

```
$ npm run test:coverage
-----------------------------|---------|----------|---------|---------|-------------------
File                         | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----------------------------|---------|----------|---------|---------|-------------------
All files                    |   83.99 |    72.85 |   77.94 |   86.33 |
 application                 |     100 |    66.66 |     100 |     100 |
  change-avatar.ts           |     100 |      100 |     100 |     100 |
  publish-event.ts           |     100 |    66.66 |     100 |     100 | 21
  publish-moment.ts          |     100 |      100 |     100 |     100 |
 domain                      |   97.89 |    83.33 |   96.15 |   99.37 |
  chat-message.ts            |     100 |      100 |     100 |     100 |
  classmate.ts               |   92.85 |    85.71 |     100 |     100 | 24,38
  domain-error.ts            |      75 |        0 |      50 |      75 | 10
  event-schedule.ts          |     100 |     87.5 |     100 |     100 | 47
  event.ts                   |     100 |      100 |     100 |     100 |
  flash-window.ts            |     100 |       80 |     100 |     100 | 8-34
  moment.ts                  |     100 |      100 |     100 |     100 |
  place.ts                   |     100 |      100 |     100 |     100 |
  ports.ts                   |       0 |        0 |       0 |       0 |
  student.ts                 |    92.3 |       80 |   85.71 |     100 | 59,65
  validation.ts              |     100 |    66.66 |     100 |     100 | 13-14
 infrastructure/cloudinary   |   39.47 |    42.85 |      40 |   41.93 |
  cloudinary-photo-storage.ts |  39.47 |    42.85 |      40 |   41.93 | 26-46
 infrastructure/local        |   74.32 |    51.85 |   67.64 |   77.41 |
  local-auth-gateway.ts      |   64.28 |       40 |   57.14 |   67.56 | 33,61-80
  local-campus-directory.ts  |       0 |      100 |       0 |       0 | 9-10
  local-change-notifier.ts   |     100 |      100 |     100 |     100 |
  local-chat-repository.ts   |   76.92 |      100 |   66.66 |      80 | 30-31
  local-event-repository.ts  |   89.47 |      100 |   85.71 |   88.23 | 35-36
  local-moment-repository.ts |   80.48 |    53.84 |   77.27 |   82.35 | 17,64-67,76
  local-photo-storage.ts     |       0 |      100 |       0 |       0 | 6
  local-report-repository.ts |       0 |      100 |       0 |       0 | 6
  local-social-graph.ts      |   77.27 |       75 |   63.63 |   88.23 | 28-29
-----------------------------|---------|----------|---------|---------|-------------------

Test Suites: 9 passed, 9 total
Tests:       56 passed, 56 total
Snapshots:   0 total
Time:        2.382 s, estimated 5 s
Ran all test suites.
```

Figura 14. Reporte de cobertura por archivo.

Pruebas manuales

Plan de `docs/PRUEBAS.md`, ajustado a la v0.4.0. Se repiten con el APK de release.

#	Caso	Resultado esperado	Evidencia
1	Registrarse con un correo que no es institucional	Mensaje "Usa tu correo institucional..."	Figura 16

#	Caso	Resultado esperado	Evidencia
2	Registrarse con correo institucional	Llega el correo de verificación (modo nube)	Figuras 19 y 24
3	Abrir la pestaña Hoy antes de la ventana	Anillo "Aún no" y feed bloqueado	Figuras 29 y 35
4	Panel de admin → Abrir la ventana ahora	En 5 s se abre la ventana y el anillo cuenta 2:00	Figura 29
5	Tomar la foto	Se toma la trasera y luego la frontal sola	Figuras 29 y 30
6	Publicar	El feed se desbloquea y tu momento aparece primero	Figuras 30, 35 y 27
7	Intentar publicar otra vez	Mensaje "Ya publicaste tu momento de hoy"	Prueba automática en `moment.test.ts`
8	Publicar después de los 2 minutos	El momento queda marcado como "Tarde"	Figura 27 (`late: true`) y prueba en `publish-moment.test.ts`
9	Reaccionar a un momento	El corazón cambia al instante y se conserva	Figuras 31 y 27
10	Usar la app sin internet	Mensajes claros, sin cierres inesperados	Pendiente: probar sin internet en el teléfono

10 Mecanismos de seguridad

La seguridad de Flashly se aplica en dos lados: la app valida para dar mensajes claros, y el servidor (reglas de Firestore y Storage) vuelve a validar todo, así un cliente modificado no puede saltarse las reglas.

Mecanismo	Implementación	Archivo
Autenticación	Firebase Authentication con correo y contraseña de al menos 8 caracteres	`firebase-auth-gateway.ts`
Correo verificado	Se envía el enlace al registrarse; sin verificar no se lee el feed ni el chat. Al verificar se pide un token nuevo	`firebase-auth-gateway.ts` , `verify.tsx`
Solo comunidad UTSV	Dominios `@alumnos.utsv.edu.mx` y `@utsv.edu.mx` , validados con Zod en la app y con expresiones regulares en el servidor	`student.ts` , `firestore.rules`

Mecanismo	Implementación	Archivo
Autorización por documento	Cada quien escribe solo lo suyo, un momento por día, de un momento ajeno solo cambian las reacciones y nadie se da permisos de admin	`firestore.rules`
"Si no publicas, no ves" en el servidor	`get` y `list` de momentos exigen que exista tu momento del día	`firestore.rules`
Validación de datos	Campos permitidos, longitudes máximas, fechas coherentes y `createdAt == request.time` en el chat	`firestore.rules`
Archivos	Solo JPEG de menos de 5 MB, cada quien en su carpeta; portadas solo admins	`storage.rules`
Datos en tránsito	HTTPS con Firebase y Cloudinary; la regla `isHttpsUrl` rechaza URLs que no sean `https://`	`firestore.rules`
Secretos fuera del repositorio	`.env`, llaves de firma (`*.jks`, `*.p12`, `*.key`, `*.pem`) y `.claude/` están en `.gitignore`	`.gitignore`
Permisos mínimos	Android solo pide `CAMERA`; `RECORD_AUDIO` está bloqueado y el micrófono desactivado	`app.json`
Retención mínima	Cada momento guarda `expiresAt` a 24 h y la app deja de mostrarlo al vencer	`firestore-moment-repository.ts`
Moderación	Reportes anónimos, borrado por admins y Cloud Function con Cloud Vision SafeSearch	`firestore.rules`, `functions/src/index.ts`
Mensajes de error seguros	Los códigos de Firebase se traducen a textos claros sin exponer detalles internos	`firebase-auth-gateway.ts`



Figura 15. Reglas de Firestore: solo cuentas UTSV verificadas, un momento por día con id `día_usuario` y feed visible solo si ya publicaste.

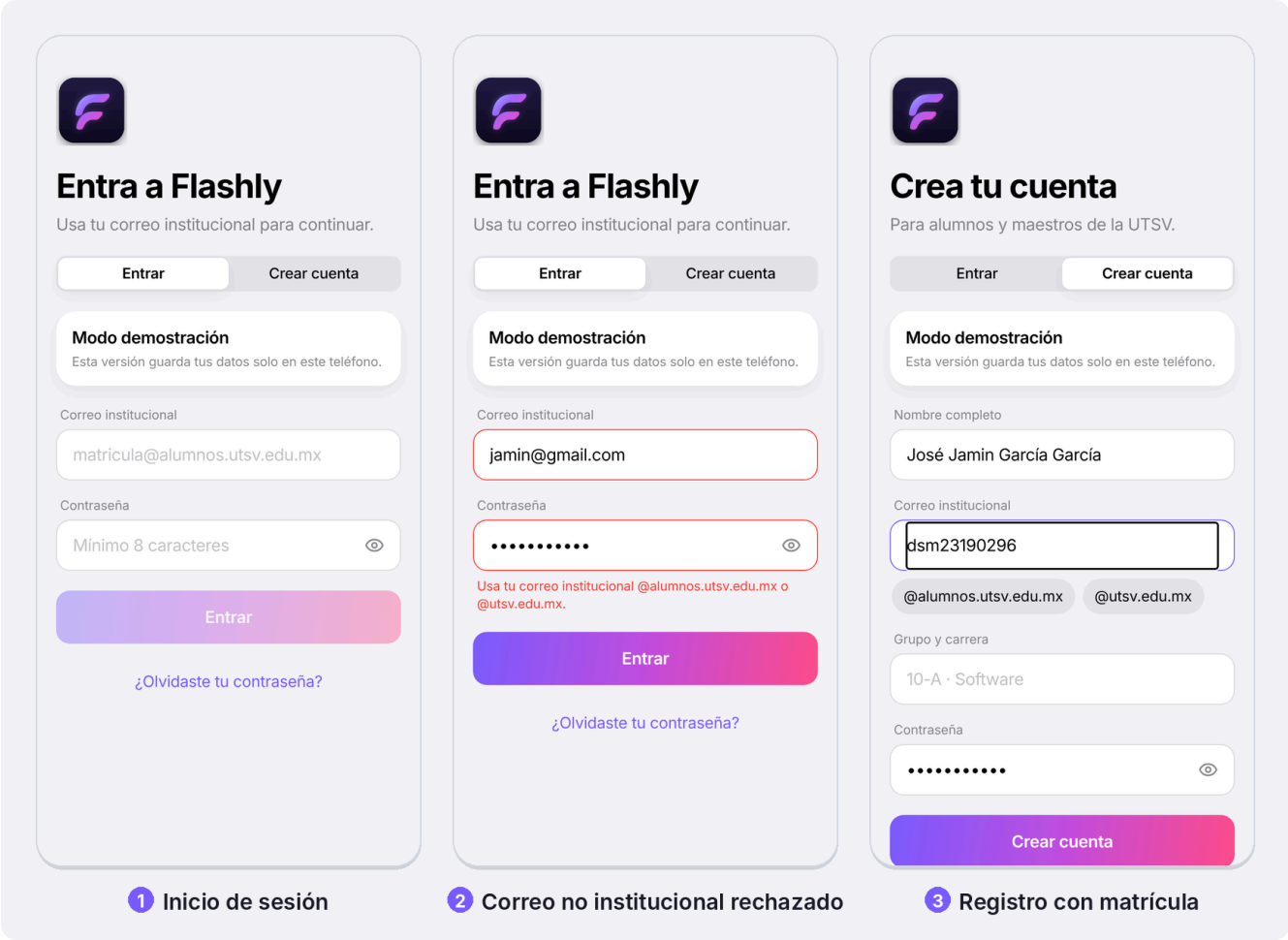


Figura 16. En la app: inicio de sesión, rechazo de un correo de Gmail y registro con autocompletado del dominio institucional.

src/domain/student.ts — Dominio · validación con Zod

```ts
export const INSTITUTION_DOMAIN = 'alumnos.utsv.edu.mx';

  …
const institutionalEmail = z
  .string()
  .trim()
  .toLowerCase()
  .email('Escribe un correo válido.')
  .refine(
    (email) => email.endsWith('@' + INSTITUTION_DOMAIN) || email.endsWith('@' + TEACHER_DOMAIN),
    `Usa tu correo institucional @${INSTITUTION_DOMAIN} o @${TEACHER_DOMAIN}.`,
  );

const password = z.string().min(8, 'La contraseña necesita al menos 8 caracteres.');

export const signInSchema = z.object({ email: institutionalEmail, password });

export const signUpSchema = signInSchema.extend({
  name: z.string().trim().min(3, 'Escribe tu nombre completo.').max(PROFILE_LIMITS.name, 'El nombre es demasiado largo.'),
  group: z.string().trim().max(PROFILE_LIMITS.group, 'El grupo es demasiado largo.'),
});

export const emailSchema = z.object({ email: institutionalEmail });
```

Figura 17. Esquemas de Zod que validan correo institucional, contraseña y nombre antes de llamar a Firebase.

firebase/storage.rules — Reglas de seguridad · Storage

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    // Solo se usa si eliges Firebase Storage en lugar de Cloudinary (requiere el plan Blaze).
    function verified() {
      return request.auth != null && request.auth.token.email_verified == true;
    }

    function smallJpeg() {
      return request.resource.size < 5 * 1024 * 1024 && request.resource.contentType.matches('image/jpeg');
    }

    // Fotos del momento: solo la dueña sube sus dos fotos.
    match /flashly/{day}/{uid}/{file} {
      allow read: if verified();
      allow write: if verified() && request.auth.uid == uid && file in ['back.jpg', 'front.jpg'] && smallJpeg();
    }

    // Foto de perfil.
    match /flashly/avatars/{uid}/{file} {
      allow read: if verified();
      allow write: if verified() && request.auth.uid == uid && smallJpeg();
    }

    // Portadas de eventos: solo administradores.
    match /flashly/events/{file} {
      allow read: if verified();
      allow write: if verified() && smallJpeg()
        && firestore.get(/databases/(default)/documents/users/$(request.auth.uid)).data.get('isAdmin', false) == true;
    }
```

Figura 18. Reglas de Storage: cuenta verificada, JPEG de menos de 5 MB y cada quien en su carpeta.

src/infrastructure/firebase/firebase-auth-gateway.ts — Adapter · Firebase Authentication

```ts
const AUTH_MESSAGES: Record<string, string> = {
  'auth/invalid-credential': 'Correo o contraseña incorrectos.',
  'auth/wrong-password': 'Correo o contraseña incorrectos.',
  'auth/user-not-found': 'No encontramos una cuenta con ese correo.',
  'auth/email-already-in-use': 'Ese correo ya tiene una cuenta. Inicia sesión.',
  'auth/weak-password': 'La contraseña necesita al menos 8 caracteres.',
  'auth/too-many-requests': 'Demasiados intentos. Espera unos minutos.',
  'auth/network-request-failed': 'Sin conexión. Revisa tu internet.',
};

const toDomainError = (error: unknown): DomainError => {
  const code = (error as { code?: string })?.code ?? '';
  return new DomainError(AUTH_MESSAGES[code] ?? 'No pudimos completar la operación. Intenta de nuevo.', code || 'auth');
};
  …
  async signUp({ name, group, email, password }: SignUpInput): Promise<Student> {
    this.signingUp = true;
    try {
      const { user } = await createUserWithEmailAndPassword(this.auth, email, password);
      await setDisplayName(user, { displayName: name });
      await this.createProfile(user, name, group);
      await sendEmailVerification(user);
      return { uid: user.uid, email, name, group, verified: false };
    } catch (error) {
      throw toDomainError(error);
    } finally {
      this.signingUp = false;
    }
  }

  …
  async refreshVerification(): Promise<boolean> {
    const user = this.auth.currentUser;
    if (!user) return false;
    await reload(user);
    // Token nuevo para que las reglas de seguridad vean el correo verificado.
    if (user.emailVerified) await user.getIdToken(true);
    return user.emailVerified;
  }
```

Figura 19. Registro con envío de verificación, traducción de errores y token nuevo al verificar el correo.

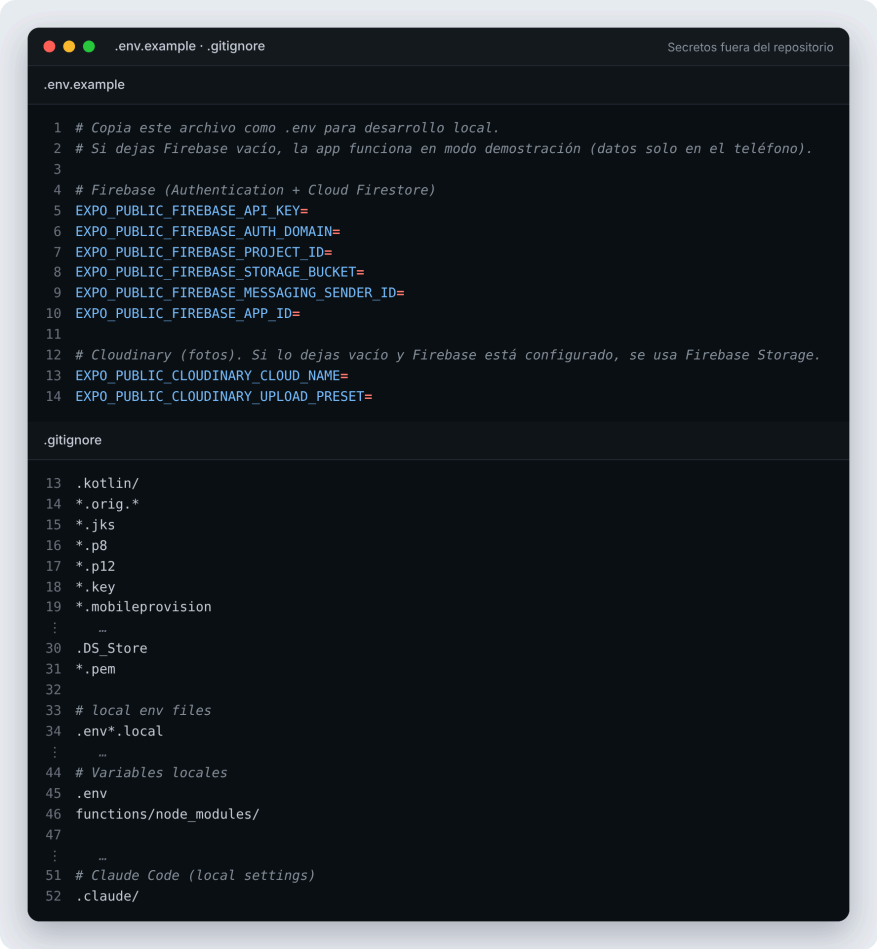


Figura 20. `.env.example` y las líneas de `.gitignore` que mantienen fuera del repositorio las variables locales y las llaves de firma.

Configuración pública. La configuración web de Firebase y el nombre del preset de Cloudinary que lleva `eas.json` no son secretos: identifican el proyecto, y la protección la dan las reglas del servidor y las restricciones del preset. Dos límites conocidos quedan en la hoja de ruta: firmar las subidas a Cloudinary y activar la política TTL de Firestore, que requiere el plan Blaze.

11 APIs de terceros

La API de terceros principal es la Upload API de Cloudinary, que guarda las dos fotos de cada momento, las fotos de perfil y las portadas de eventos. La app también consume Google Cloud Vision desde el servidor y varias APIs nativas del teléfono a través de Expo.

API	Proveedor	Para qué la usa Flashly	Dónde
Upload API (`POST /v1_1/<cloud>/image/upload`)	Cloudinary	Subir fotos con un preset sin firma y recibir su `secure_url`	`cloudinary-photo-storage.ts`

API	Proveedor	Para qué la usa Flashly	Dónde
Cloud Vision, SafeSearch	Google Cloud	Revisar cada foto publicada y ocultar las inapropiadas	`functions/src/index.ts`
Firebase JS SDK	Google Firebase	Autenticación, base de datos en tiempo real y archivos	`src/infrastructure/firebase/`
Camera, Image Manipulator, Image Picker	Expo	Cámara trasera y frontal, compresión a 1080/540 px y galería	`capture.tsx`, `src/infrastructure/device/`
Notifications	Expo	Aviso local del flash y recordatorios 30 min antes de un evento	`expo-reminder-scheduler.ts`

Cómo funciona la subida a Cloudinary

1. El caso de uso pide `photos.upload(foto, { folder: 'flashly/<día>/<uid>', name: 'back' })`.
2. El adaptador arma un `FormData` con el archivo JPEG, el `upload_preset` y la carpeta.
3. En Android usa `XMLHttpRequest` con 60 s de límite, porque maneja bien las rutas locales del teléfono.
4. Cloudinary responde con `secure_url`; esa URL `https` es la que se guarda en Firestore.
5. Cada falla se traduce a un error del dominio: sin conexión, tiempo agotado, foto rechazada o respuesta sin URL.

src/infrastructure/cloudinary/cloudinary-photo-storage.ts — API de terceros · Cloudinary

```ts
/**
 * API de terceros: sube fotos a Cloudinary con un "upload preset" sin firma.
 * Usa FileSystem.uploadAsync (nativo) para evitar problemas con URIs locales en Android.
 * En tests se puede inyectar un http alternativo para no depender de la red.
 */
export class CloudinaryPhotoStorage implements PhotoStorage {
  constructor(private readonly config: CloudinaryConfig, private readonly http?: typeof fetch) {}

  get endpoint(): string {
    return `https://api.cloudinary.com/v1_1/${this.config.cloudName}/image/upload`;
  }

  async upload(localUri: string, target: PhotoTarget): Promise<string> {
    return this.http
      ? this.uploadWithFetch(localUri, target)
      : this.uploadWithFileSystem(localUri, target);
  }

  private uploadWithFileSystem(localUri: string, target: PhotoTarget): Promise<string> {
    return new Promise((resolve, reject) => {
      const xhr = new XMLHttpRequest();
      xhr.open('POST', this.endpoint);
      xhr.timeout = 60000;
      xhr.onload = () => {
        let body: CloudinaryResponse;
        try { body = JSON.parse(xhr.responseText) as CloudinaryResponse; } catch { body = {}; }
        if (xhr.status >= 400)
          return reject(new DomainError(body.error?.message ?? 'Cloudinary rechazó la foto.', 'upload-rejected'));
        if (!body.secure_url)
          return reject(new DomainError('Cloudinary no devolvió la dirección de la foto.', 'upload-invalid'));
        resolve(body.secure_url);
      };
      xhr.onerror = () => reject(new DomainError('Sin conexión al subir la foto (XHR).', 'upload-network'));
      xhr.ontimeout = () => reject(new DomainError('Tiempo agotado al subir la foto.', 'upload-timeout'));

      const form = new FormData();
      form.append('file', { uri: localUri, type: 'image/jpeg', name: `${target.name}.jpg` } as unknown as Blob);
      form.append('upload_preset', this.config.uploadPreset);
      form.append('folder', target.folder);
      xhr.send(form);
    });
  }
```

Figura 21. Adaptador de Cloudinary: implementa el puerto `PhotoStorage`, así el resto de la app no sabe qué proveedor guarda las fotos. En pruebas se le inyecta un `fetch` simulado.

Las cuentas se configuran con las variables `EXPO_PUBLIC_CLOUDINARY_CLOUD_NAME` y `EXPO_PUBLIC_CLOUDINARY_UPLOAD_PRESET`. Si no existen y Firebase sí, la fábrica cambia sola a Firebase Storage (patrón Strategy).

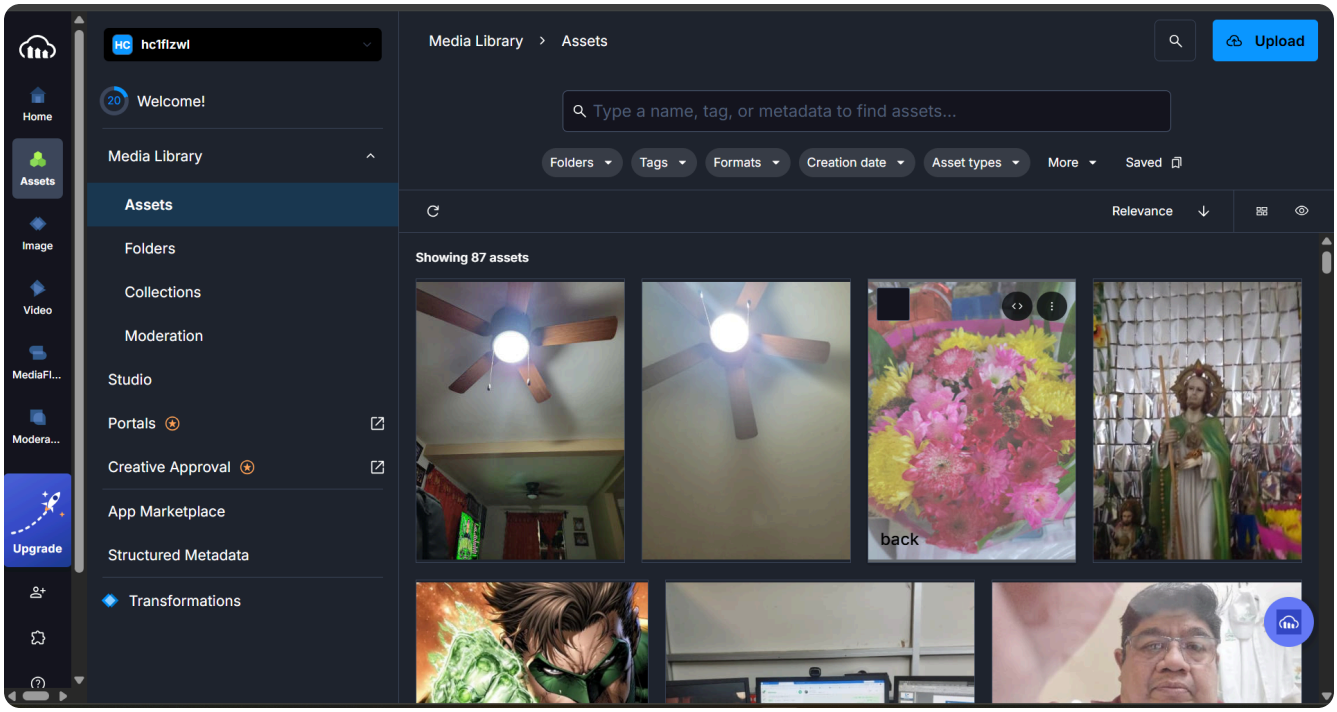


Figura 22. Media Library de la cuenta de Cloudinary `hc1flzwl` con 87 archivos, entre ellos fotos de momentos (como la marcada "back") y fotos de perfil subidas desde la app.

12 Servicios en la nube

Flashly usa el proyecto de Firebase `flashly-d0034` como backend: Authentication para las cuentas y Cloud Firestore como base de datos en tiempo real. Las fotos van a Cloudinary, el APK se compila en Expo EAS y la CI corre en GitHub Actions.

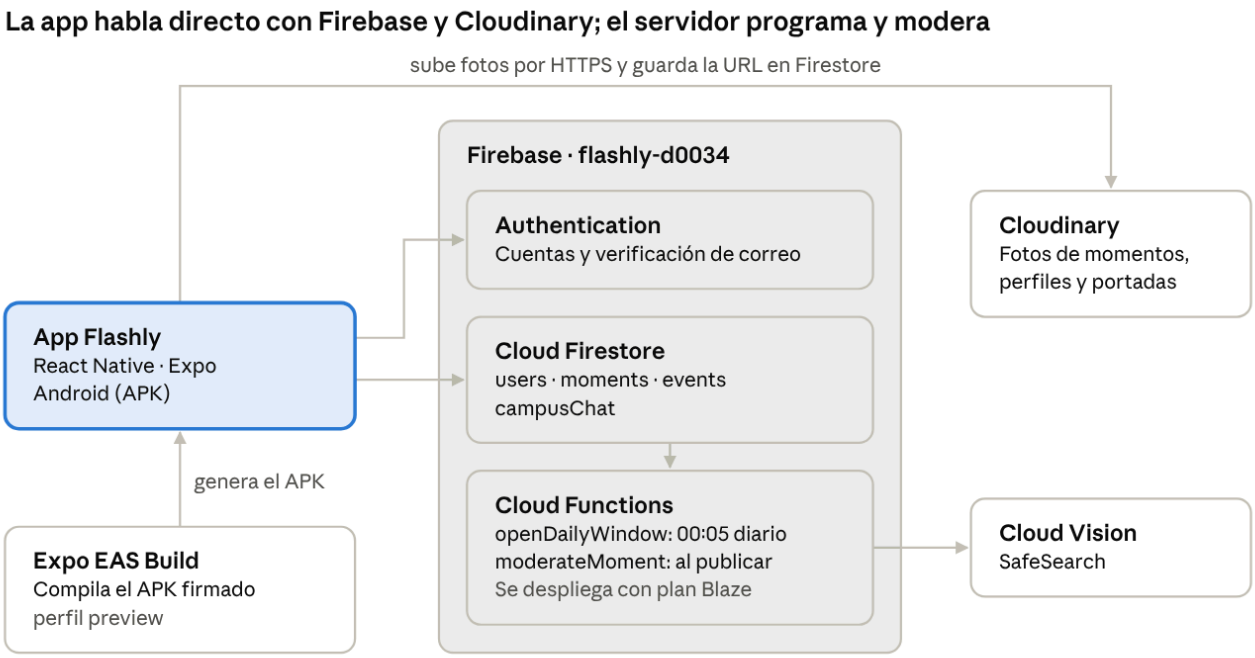


Figura 23. La app se conecta directo a Firebase y a Cloudinary; las Cloud Functions registran la ventana del día y moderan cada momento con Cloud Vision.

Servicio	Proveedor	Plan	Uso en Flashly
Firebase Authentication	Google	Gratuito (Spark)	Registro con correo institucional, verificación y recuperación de contraseña
Cloud Firestore	Google	Gratuito (Spark)	Perfiles, momentos, eventos, chat y reportes con actualización en tiempo real; índice compuesto `windowId` + `createdAt`
Cloud Functions v2	Google	Blaze (pago por uso)	`openDailyWindow` registra la hora sorpresa a las 00:05; `moderateMoment` revisa las fotos con Cloud Vision
Cloudinary	Cloudinary	Gratuito	Almacenamiento y entrega de fotos por HTTPS
Expo EAS Build	Expo	Gratuito	APK de prueba (perfil `preview`) del proyecto `cca195dd-c0e0-4562-a2e1-de2a7bc1b65b`
GitHub Actions	GitHub	Gratuito para repos públicos	Tipos y pruebas en cada push y Pull Request

Evidencia en la consola de Firebase. El proyecto `flashly` corre en el plan Spark (gratuito) y ya tiene cuentas y datos reales del equipo y del docente.

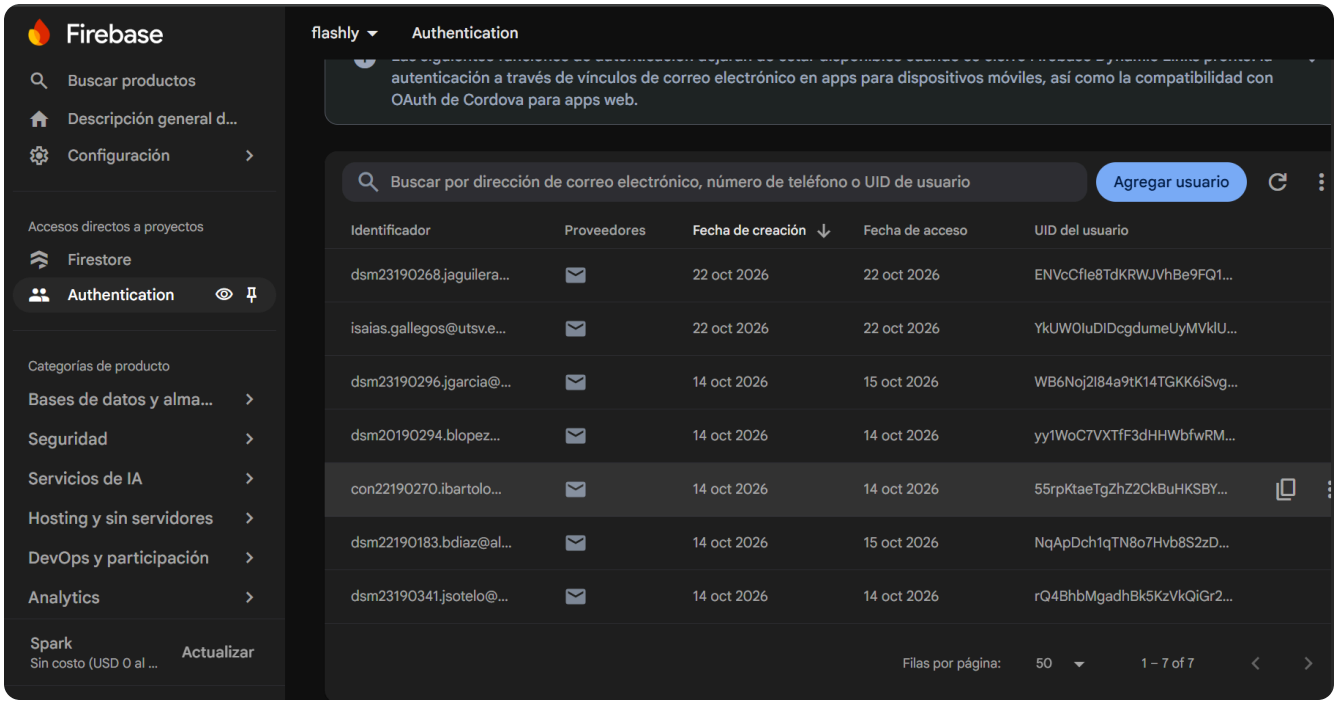


Figura 24. Firebase Authentication: 7 cuentas con correo y contraseña, todas con dominio institucional, entre ellas las del equipo y la del docente.

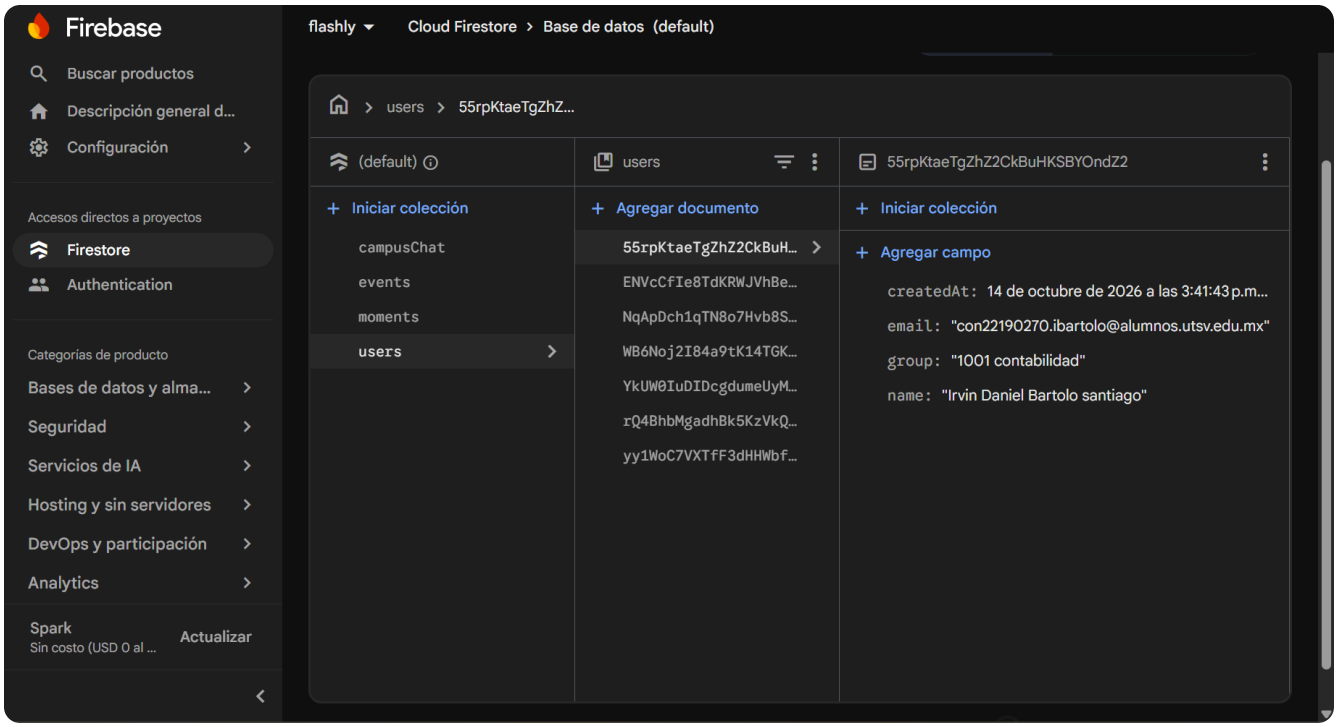


Figura 25. Cloud Firestore con las colecciones `campusChat`, `events`, `moments` y `users`. Cada perfil guarda solo los campos que permite la regla `validProfile`.

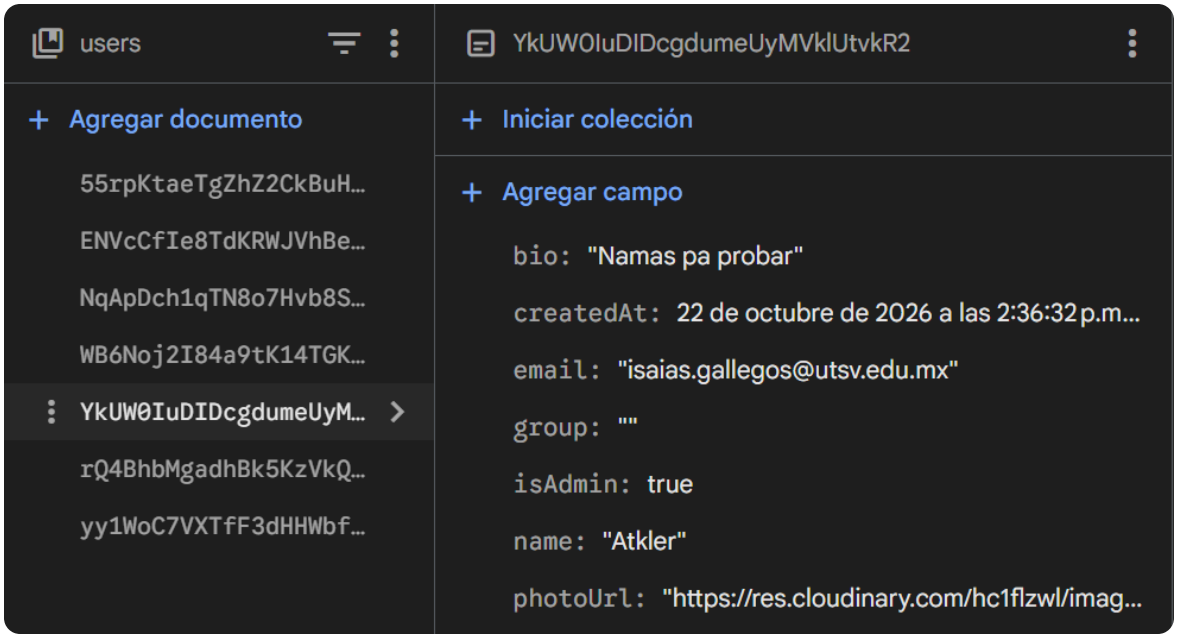


Figura 26. Perfil del docente con `isAdmin: true`, asignado desde la consola porque las reglas impiden que alguien se lo dé desde la app, y `photoUrl` alojada en Cloudinary.

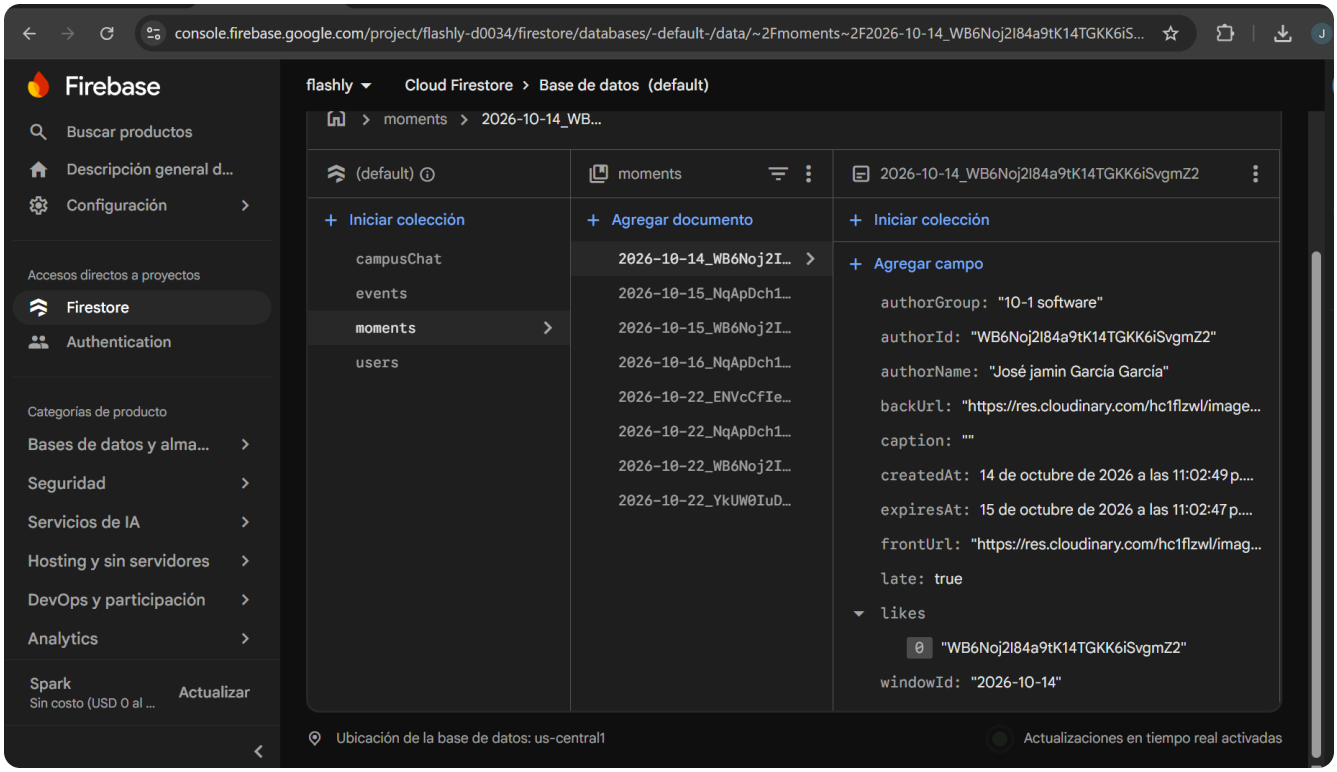


Figura 27. Documento real de `moments`: id `2026-10-14_<uid>` (un momento por día), fotos `backUrl` y `frontUrl` en Cloudinary, `expiresAt` 24 h después de `createdAt`, `late: true` por publicar fuera de la ventana y una reacción en `likes`. La base corre en us-central1 con actualizaciones en tiempo real.

Las reglas, los índices y las funciones se despliegan con Firebase CLI (`firebase deploy --only firestore`), según `firebase.json`. Las Cloud Functions están escritas y listas en `functions/`, pero solo se pueden desplegar con el plan Blaze; con el plan gratuito la app calcula la misma hora sorpresa en el teléfono con el mismo algoritmo.

functions/src/index.ts — Cloud Functions

```ts
/** Registra la ventana del día. Corre todos los días a las 00:05 hora del centro de México. */
export const openDailyWindow = onSchedule({ schedule: '5 0 * * *', timeZone: TIME_ZONE }, async () => {
  const day = todayId();
  const minute = hash('flashly-' + day) % ((TO_HOUR - FROM_HOUR) * 60);
  const startMinute = FROM_HOUR * 60 + minute;
  await getFirestore().doc(`windows/${day}`).set({
    day,
    startMinute,
    durationMinutes: WINDOW_MINUTES,
    createdAt: FieldValue.serverTimestamp(),
  });
  logger.info('Ventana del día registrada', { day, hora: `${Math.floor(startMinute / 60)}:${String(startMinute % 60).padStart(2,
});
  …
/**
 * Moderación con Cloud Vision (SafeSearch): revisa las dos fotos por su URL pública
 * (Cloudinary o Firebase Storage). Si alguna es inapropiada, oculta el momento borrando el documento.
 */
export const moderateMoment = onDocumentCreated('moments/{momentId}', async (event) => {
  const data = event.data?.data();
  if (!data) return;
  for (const url of [data.backUrl, data.frontUrl].filter(Boolean)) {
    const [result] = await vision.safeSearchDetection(String(url));
    const s = result.safeSearchAnnotation;
    const flagged = !!s && [s.adult, s.violence, s.racy].some((level) => BLOCKED.includes(String(level)));
    if (!flagged) continue;
    await event.data?.ref.delete();
    await getFirestore().collection('moderation').add({
      momentId: event.params.momentId,
      authorId: data.authorId,
      reason: 'safesearch',
      urls: [data.backUrl, data.frontUrl],
      at: FieldValue.serverTimestamp(),
    });
    logger.warn('Momento ocultado por moderación', { momentId: event.params.momentId });
    return;
  }
});
```

Figura 28. Cloud Functions: tarea programada que registra la ventana del día y disparador que modera cada momento nuevo con SafeSearch.

13 Aplicación en funcionamiento

El recorrido completo de la v0.4.0 funciona de punta a punta: registro, flash del día, doble cámara, publicación, feed desbloqueado, comunidad y administración. Las capturas siguen ese orden y se tomaron en una sola sesión en modo demostración, donde los compañeros, eventos y mensajes de ejemplo son ficticios; en modo nube esos datos vienen de Firestore (Figuras 24 a 27). Al final de la sección están las mismas pantallas en un teléfono Android con el APK y datos reales (Figuras 35 y 36).

El flash del día. Antes de la hora sorpresa el anillo dice "Aún no" y el feed está bloqueado. Al abrirse la ventana empieza la cuenta regresiva de 2 minutos y el botón lleva a la cámara.

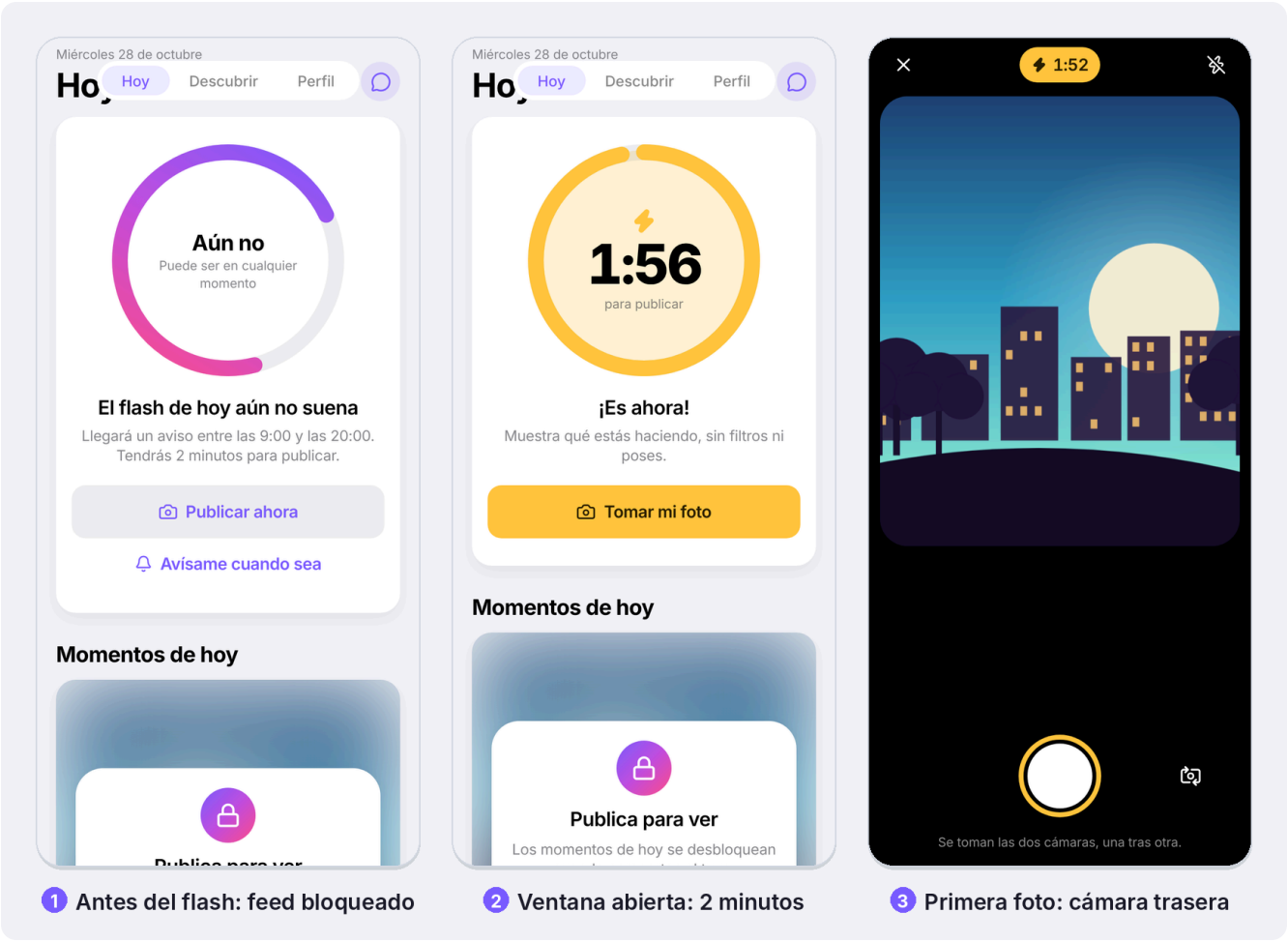


Figura 29. De izquierda a derecha: feed bloqueado antes del flash, ventana abierta con 1:56 restantes y primera foto con la cámara trasera.

Publicar. La segunda foto se toma sola con la cámara frontal. En la revisión se elige el lugar del campus y se escribe un comentario opcional; al publicar, el feed se desbloquea y el momento propio aparece primero.

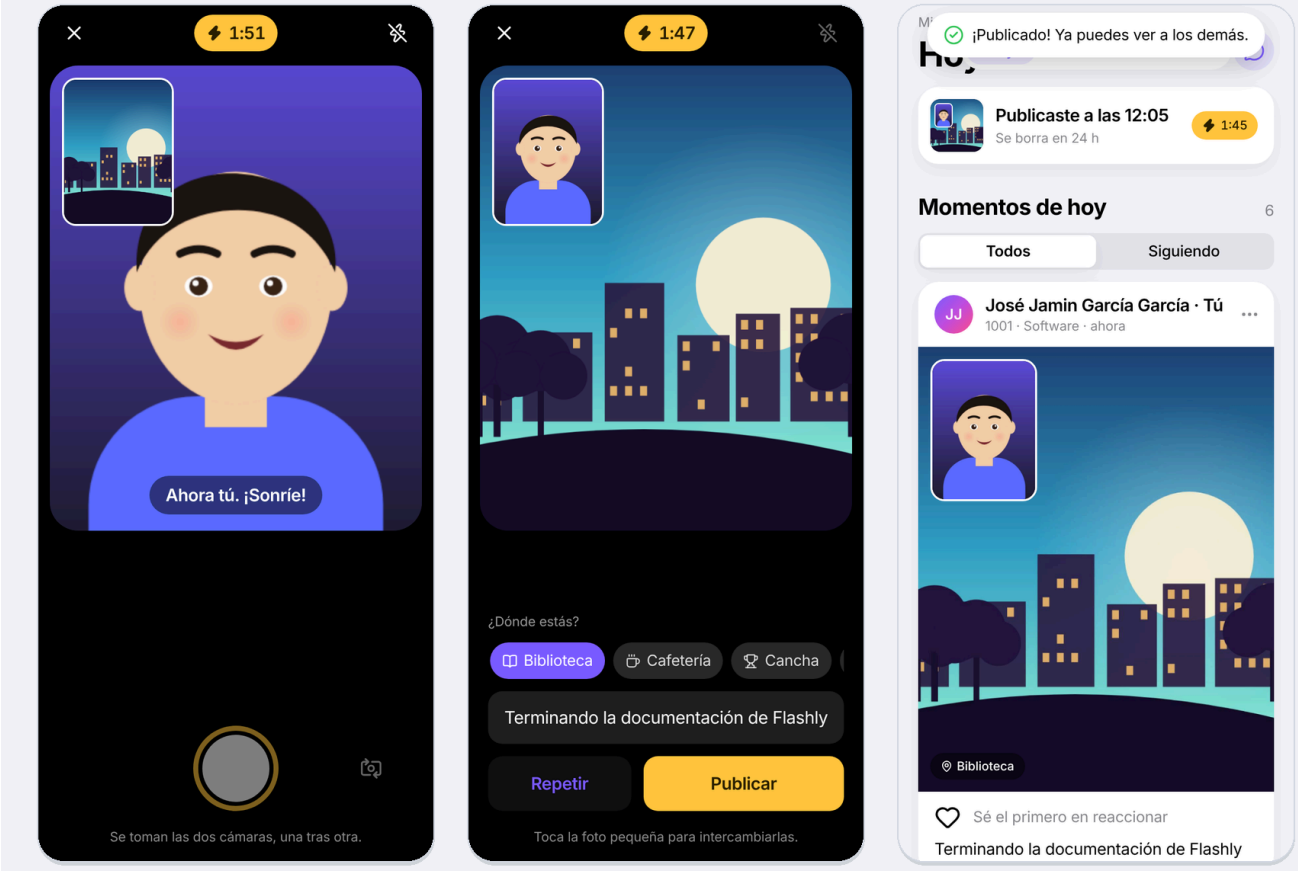


Figura 30. Foto frontal automática, revisión con lugar "Biblioteca" y feed desbloqueado con el aviso "¡Publicado!".

El feed y los compañeros. Las reacciones cambian al instante, cada lugar agrupa los momentos publicados desde ahí y el perfil de un compañero muestra su momento del día y el botón para seguirlo.



Figura 31. Reacción con "2 me gusta", momentos de la Biblioteca y perfil de Sofía Cruz.

Descubrir y chat. La pestaña Descubrir reúne eventos, lugares con movimiento y personas. Cada evento permite activar un recordatorio 30 minutos antes, y el chat del campus es abierto para alumnos y maestros verificados.



Figura 32. Eventos y lugares, detalle del "Torneo relámpago de fútbol" y chat del campus con un mensaje enviado.

Perfil y administración. El perfil concentra la apariencia, el idioma, las notificaciones y la cuenta. Las cuentas con permiso de admin ven el panel para crear eventos con portada, borrar eventos activos y abrir la ventana del flash al momento para hacer pruebas.

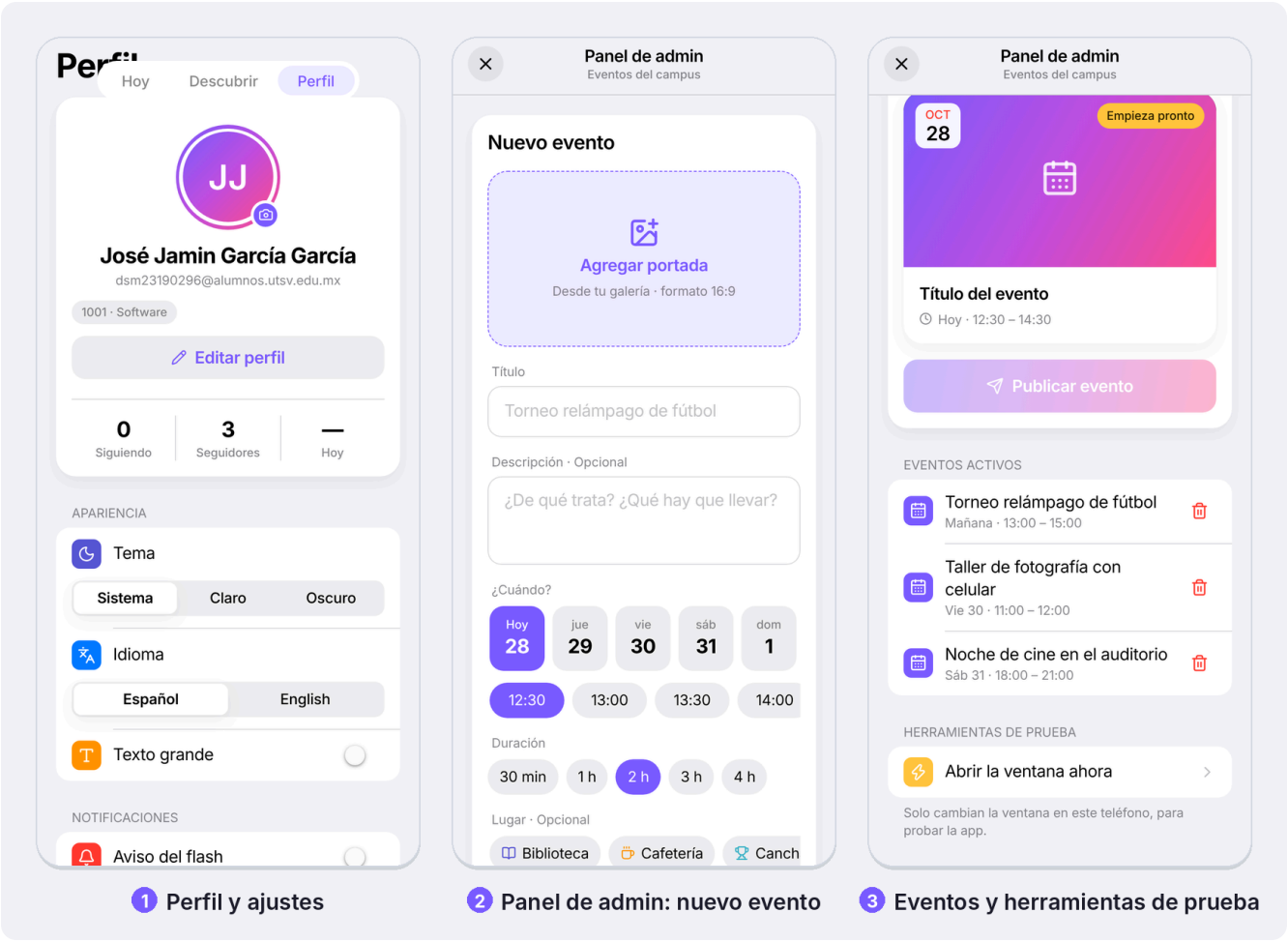
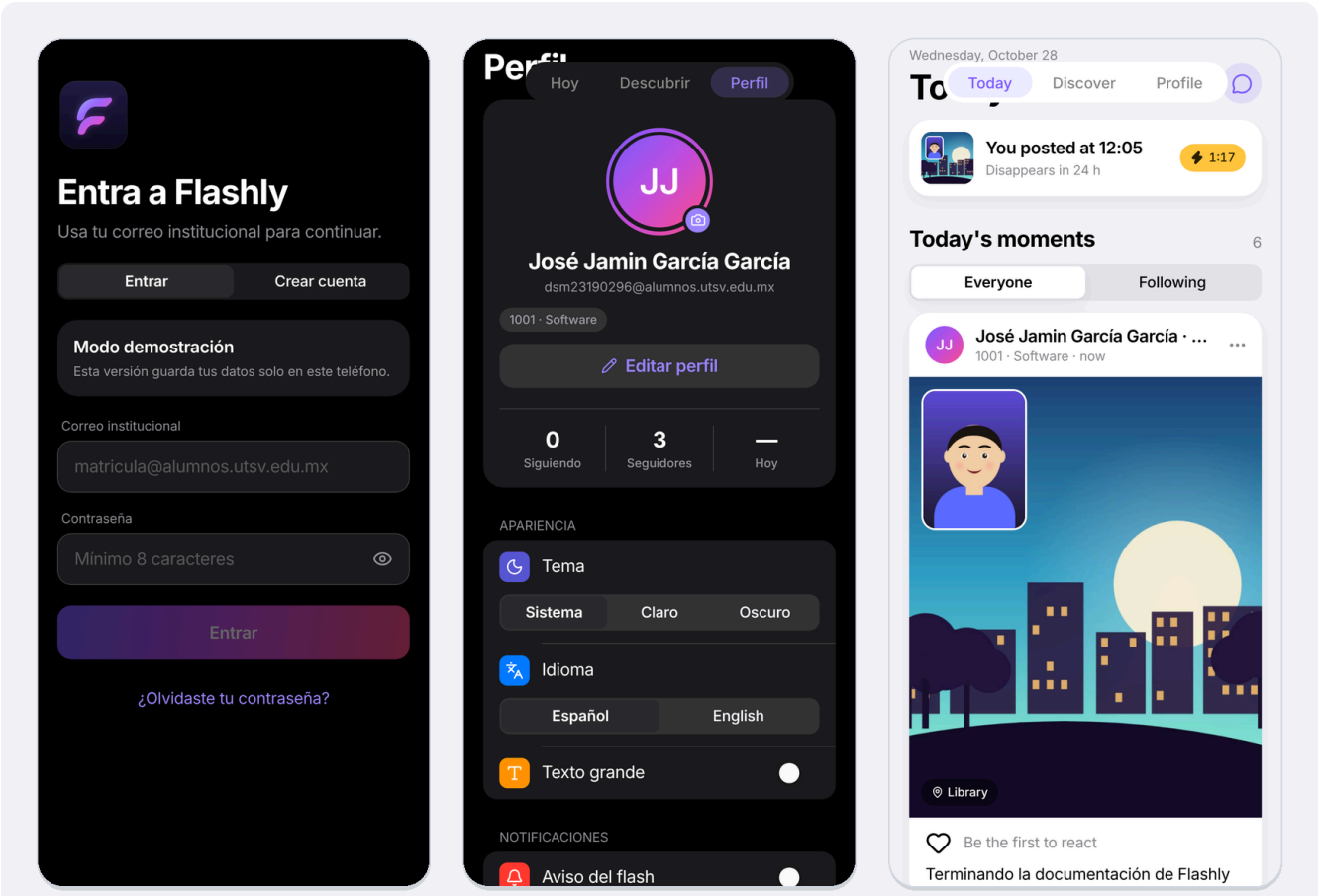


Figura 33. Perfil con ajustes, formulario de nuevo evento y lista de eventos activos con las herramientas de prueba.

Accesibilidad e idiomas. La app sigue el tema del sistema (claro u oscuro), ofrece texto grande y cambia toda la interfaz entre español e inglés.



1 Tema oscuro 2 Perfil en tema oscuro 3 Interfaz en inglés

Figura 34. Inicio de sesión y perfil en tema oscuro, y la pestaña Hoy con la interfaz en inglés.

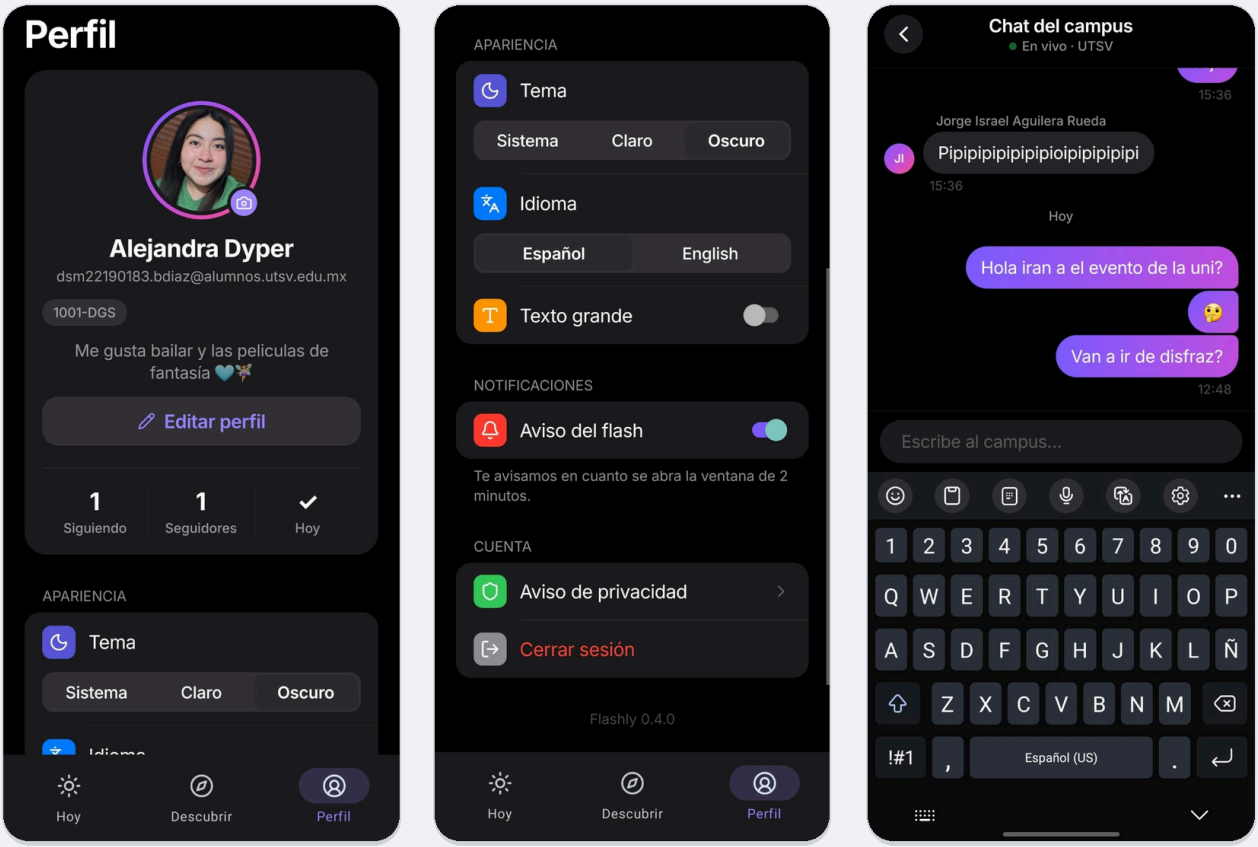
En un teléfono Android real

Las mismas pantallas corren en Android con el APK de la v0.4.0 y datos reales de Firestore, en la cuenta de Bárbara Alejandra Díaz Pérez. Aquí las pestañas aparecen abajo, como es nativo en Android.



1 Antes del flash: feed bloqueado 2 Momento publicado: feed abierto 3 Descubrir: lugar con 1 momento

Figura 35. APK en Android con datos reales: feed bloqueado antes del flash, momento publicado con foto trasera y frontal, y la Cancha con 1 momento en Descubrir.



1 Perfil real en la nube 2 Ajustes: tema, idioma, aviso 3 Chat del campus en vivo

Figura 36. Perfil real con foto, grupo 1001-DGS y 1 seguidor; ajustes con tema oscuro y aviso del flash activo (Flashly 0.4.0); y chat del campus con mensajes del equipo.

14 Enlace del repositorio en funcionamiento

REPOSITORIO

https://github.com/Jamgargar04/flashly

v0.4.0 en la rama develop · ejecuciones de CI

El repositorio es público y funciona desde cero. El 28 de octubre de 2026 se clonó en un equipo limpio, se instaló con `npm ci`, pasó la revisión de tipos y las 56 pruebas, y `expo export` generó la versión web con sus 19 rutas; de esa compilación salen las capturas de la app de este documento.

APK en funcionamiento. La misma versión está compilada como APK en Expo EAS y se instala en cualquier teléfono Android desde el enlace o escaneando el código QR:

```
expo.dev/accounts/jamingarciadev/projects/flashly/builds/7fc55cc5-2446-4d05-aba1-2152da852200
```

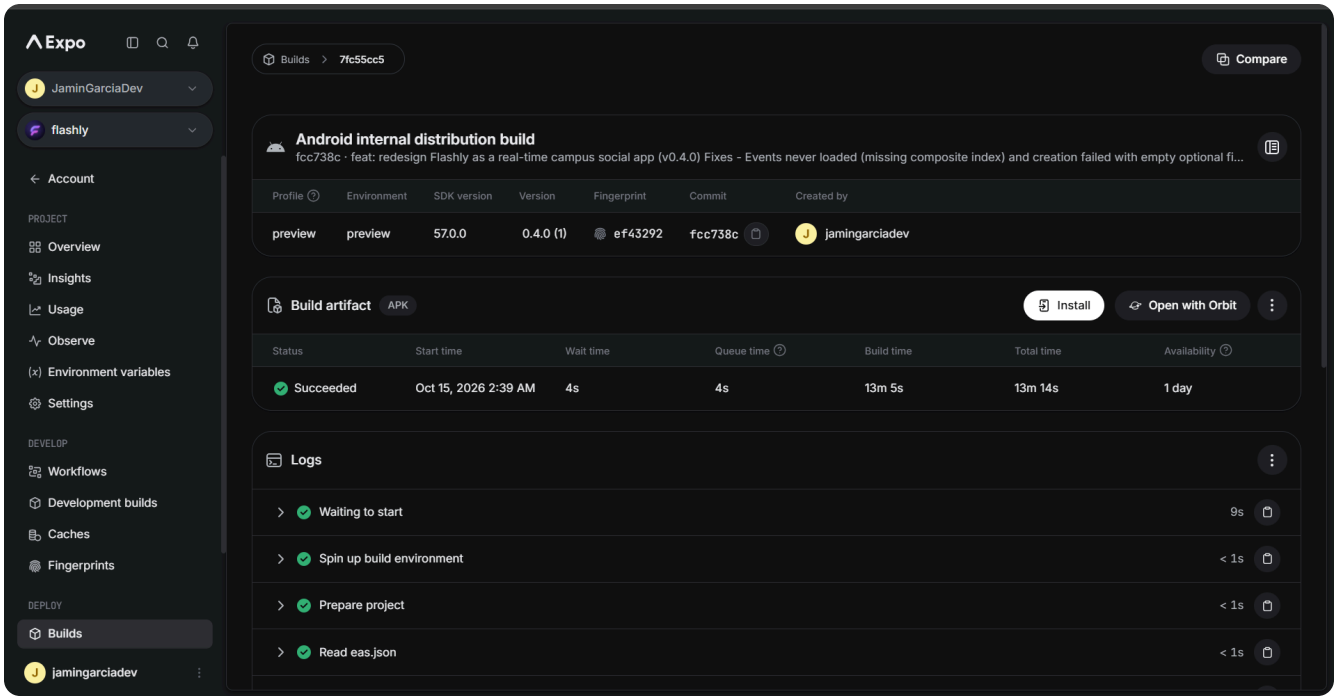


Figura 37. Build de EAS para Android (distribución interna) del commit `fcc738c`: perfil `preview`, SDK 57.0.0, versión 0.4.0 (1), terminado con éxito el 15 de octubre de 2026 en 13 min 14 s.

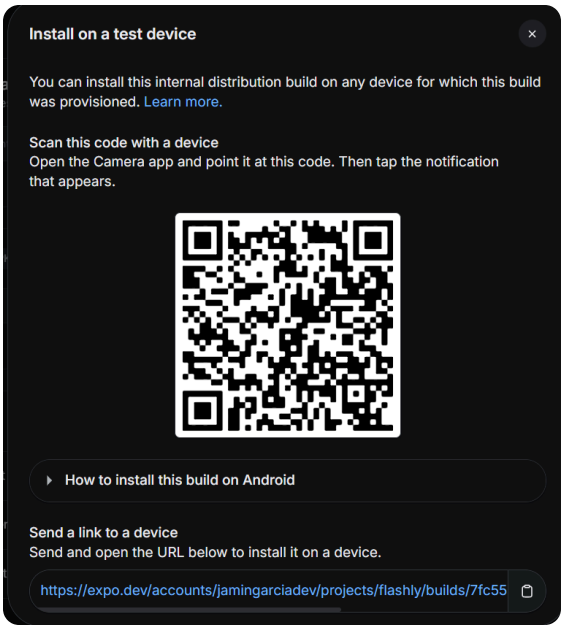


Figura 38. Código QR para instalar el APK en un teléfono Android.

Probarlo en 5 minutos (modo demostración, sin cuentas en la nube)

```
git clone https://github.com/Jamgargar04/flashly.git
cd flashly
git checkout develop
npm install
npx expo start
```

Para no esperar la hora sorpresa: Perfil → Panel de admin → Abrir la ventana ahora (en modo demostración toda cuenta es admin).

Modo nube y APK

1. Copiar `.env.example` como `.env` y llenar las variables de Firebase y Cloudinary.
2. Desplegar reglas e índices con `firebase deploy --only firestore`.
3. Generar el APK con `npx eas-cli@latest build -p android --profile preview`; al terminar, EAS entrega el enlace de descarga.

Comando	Qué hace
`npm start`	Servidor de desarrollo
`npm run typecheck`	Revisión de tipos con TypeScript
`npm test`	Pruebas automáticas con Jest
`npm run test:coverage`	Pruebas con reporte de cobertura
`npm run check`	Tipos y pruebas, lo mismo que corre la CI

15 Conclusiones

Flashly v0.4.0 cumple el resultado de aprendizaje: es una aplicación móvil funcional, construida sobre un caso de estudio real de la UTSV y versionada en un repositorio público con integración continua.

Clean Architecture y los patrones Repository, Strategy y Adapter permitieron conectar Firebase y Cloudinary sin tocar las reglas del negocio, y la inyección de dependencias hizo posible probar esa lógica con 56 pruebas automáticas. La consola de Firebase confirma el uso real: siete cuentas institucionales, cuatro colecciones con datos y un administrador asignado desde el servidor, y el build de EAS confirma que el APK de la v0.4.0 compila, se instala y funciona en Android con datos reales.

La seguridad no depende solo de la app: las reglas de Firestore y Storage repiten cada validación en el servidor, incluida la regla central del producto, "si no publicas, no ves". Los siguientes pasos hacia producción son subidas firmadas a Cloudinary, pruebas de reglas en el emulador y notificaciones push desde el servidor.

Referencias del repositorio (rama develop)

- README.md
- docs/ARQUITECTURA.md
- docs/PRUEBAS.md
- docs/HOJA-DE-RUTA.md
- firebase/firestore.rules